

Faculty of Mathematics and Economics

College of Technology

National Economics University, Vietnam



FINAL REPORT

Design NUFI Personal Finance Management System

Student ID: 11245940
Full Name: Lê Thị Thuỳ Trang
Class: DSEB66B
Course: Database Management Systems
Instructor: PhD. Trần Hùng
Source Code: [GitHub Repository](#)
PDF: [Project 13](#)

Ha Noi - 5/2026

Contents

List of Figures	4
List of Tables	5
1 Introduction	7
1.1 Reasons for choosing the topic	7
1.2 The real-world problem to be solved	7
1.3 System objectives	8
1.4 Scope of the project	10
1.5 Methodology	11
1.6 Conclusion of the introduction	13
2 Problem Analysis	13
2.1 Problem description and data characteristics	13
2.2 Main actors of the system	14
2.3 Main functions of the system	15
2.4 Functional and non-functional requirements	16
2.5 Conclusion of the problem analysis chapter	18
3 Database Design	18
3.1 Conceptual data model	18
3.2 Transforming the ERD into a relational schema	20
3.3 Data normalization	23
3.4 Explanation of main tables, primary keys, foreign keys, and integrity constraints	24
3.5 Index design and query optimization	26
3.6 Advanced SQL objects in the system	28
3.6.1 VIEW	29
3.6.2 FUNCTION	30
3.6.3 PROCEDURE	30

3.6.4	TRIGGER	31
3.6.5	Overall assessment	33
3.7	Separation of logic between Database and Django	33
3.8	Conclusion	34
4	Database Implementation Using SQL	35
4.1	Introduction to the database management system used	35
4.2	Execution order of SQL files	35
4.3	Implementing table structure with DDL	37
4.4	Implementing data constraints and validation rules	39
4.5	Implementing indexes	39
4.6	Implementing VIEW, FUNCTION, PROCEDURE, and TRIGGER	40
4.7	Sample data and its testing significance	41
4.8	Conclusion	42
5	Business Rules and Their Implementation in the Database	42
5.1	Main business rules of the system	42
5.2	Separation of enforcement between SQL and Django	44
5.3	Flow for creating an expense and generating an alert	46
5.4	Why not put all logic into triggers and procedures?	47
5.5	Conclusion	47
6	Database Integration with the Application and Feature Illustration	48
6.1	Django backend and mapping to MySQL	48
6.2	Data flow from input forms to MySQL	50
6.3	Flow for updating balances, budgets, and alerts	51
6.4	Illustration of completed features	52
6.5	Conclusion	54
7	Extended Section: System Deployment and Operations	55
7.1	System runtime environments	55
7.2	System packaging and deployment	57

7.3	CronJob, backup, and monitoring	57
7.4	Roles of extended components	58
7.5	Conclusion	59
8	Testing and Evaluation	60
8.1	Functional and business testing	61
8.2	Testing SQL objects and integrity constraints	63
8.3	Evaluation of achieved results	64
8.4	Current limitations and improvement directions	66
8.5	Conclusion	68
9	Conclusion and Future Development	68
	References	71

List of Figures

1	Overview of the personal finance management problem addressed by the NUFi system	8
2	Permission model among actors in the NUFi personal finance management system	15
3	Overall architecture of the NUFi system and the position of the MySQL database in the overall processing flow	19
4	Simplified ERD of the NUFi personal finance management system	20
5	Detailed relational schema of the NUFi personal finance management system	22
6	Database installation flow from schema to sample data	36
7	Business flow for creating an expense, updating balance, checking budget, and generating an alert	46
8	Data flow from input forms to MySQL	50
9	Flow for updating balances, budgets, and alerts	51
10	Overview dashboard of the NUFi system	53
11	Income and expense management interface of the NUFi system	54
12	Overall deployment architecture of the NUFi system	56
13	CronJob, backup, and monitoring flow of the NUFi system	58
14	Screenshot showing the NUFi system running after integration	61

List of Tables

1.1	Input / Process / Output of the main subsystems	9
1.2	In-scope and Out-of-scope of the project	10
1.3	Mapping between business requirements, data tables, processing modules, and report/alert outputs	12
2.1	Actor – Permission – Function Matrix	14
2.2	Functional and Non-functional Requirements Matrix	16
3.1	Explanation of data normalization: 1NF / 2NF / 3NF	23
3.2	Analysis of primary keys, foreign keys, and integrity constraints	25
3.3	Some representative indexes in the NUFI system	27
3.4	VIEWS in the NUFI system	29
3.5	FUNCTIONs in the NUFI system	30
3.6	PROCEDUREs in the NUFI system	31
3.7	TRIGGER groups in the NUFI system	32
3.8	Separation of logic between Database and Django	34
4.1	Execution order of SQL files in the project	36
4.2	List of tables and corresponding DDL script	38
4.3	INDEX, VIEW, FUNCTION, PROCEDURE, TRIGGER list and roles	40
5.1	List of main business rules of the system	43
5.2	Mapping between business rules and enforcement layer (SQL / Django)	44
6.1	Mapping between Django modules and the data tables they use	49
6.2	List of main features and completion status	52
7.1	Comparison of system runtime environments	56
7.2	Roles of extended deployment and operational components	59
8.1	Functional test cases and results	62
8.2	SQL object test cases	64
8.3	Comparison between manual management and using the NUFI system	65

8.4	Current limitations, causes, and improvement directions	67
-----	---	----

1. Introduction

1.1 Reasons for choosing the topic

In the context of modern life, personal finance management has become a practical need for many groups such as students, office workers, freelancers, and young households. Every day, an individual may generate many different financial transactions such as receiving salary, paying for food, settling bills, shopping, saving, or repaying debts. If these transactions are not recorded and monitored systematically, users will face difficulties in controlling cash flow and evaluating their financial situation.

In practice, many people still manage their finances using manual methods such as notebooks, Excel files, or note-taking applications. Although these methods are simple, they often lack consistency, make data aggregation difficult, complicate historical lookup, and make it hard to generate accurate reports as the amount of data grows. Therefore, building a personal finance management system is necessary both in terms of practical application and academic value.

This topic was chosen because it is a familiar problem with a clear business scope, yet still complex enough to apply important contents of the Database Management Systems course such as requirements analysis, data model design, relational schema construction, SQL implementation, query organization, and integrity control [1, 2].

1.2 The real-world problem to be solved

The problem addressed in this project is to build a system that helps users manage all of their personal financial data on a unified, easy-to-use platform with accurate aggregation capability. The system must not only support recording income and expenses, but also track account balances, set budgets, detect alerts, and generate summary reports over time.

The main issues that the system needs to solve include:

- Store personal financial data in a structured manner.
- Classify transactions by category to facilitate aggregation and reporting.
- Track balances across financial accounts such as bank accounts, e-wallets, or cash.
- Manage budgets by month or by category.
- Generate alerts when spending approaches or exceeds the budget.
- Track debts and payment history.

- Support statistical reporting so users can evaluate their financial situation.

To provide an overall view of the problem, Figure 1 illustrates the relationships between users, the main financial operations, and the outputs that support decision-making such as alerts and reports.

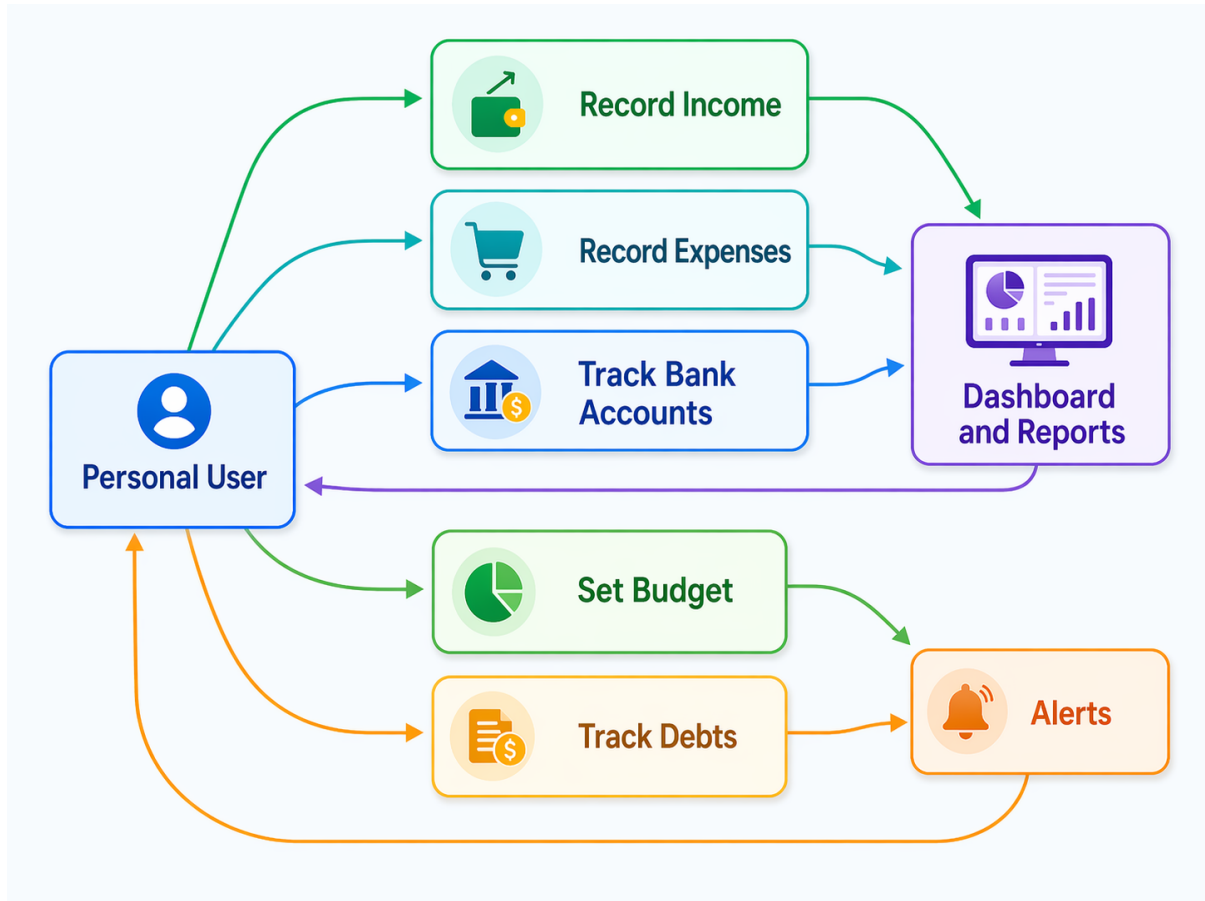


Figure 1: Overview of the personal finance management problem addressed by the NUFI system

From the diagram above, it can be seen that the system not only stores income and expense records, but also has to track financial accounts, budgets, and debts, thereby generating alerts and reports to support personal finance management in a systematic manner.

1.3 System objectives

The overall objective of the system is to build a personal finance management application that can support users in managing their financial data in a complete, accurate, and intuitive way. The system should help users record, monitor, and exploit financial data more effectively than manual methods.

The specific objectives include:

- Build a relational database suitable for the personal finance management problem.
- Support the management of user profiles, financial accounts, income, expenses, budgets, debts, and alerts.
- Provide summary reports by day, month, year, and category.
- Ensure that data is stored consistently, with clear constraints and extensibility.
- Demonstrate the central role of MySQL in a complete software system.

To clarify the operating flow of the main subsystems, the system is described in the form of an Input – Process – Output model as shown in Table 1.1.

Table 1.1: **Input / Process / Output of the main subsystems**

Subsystem	Input	Process	Output
Account and authentication	Registration information, login information, personal profile	Authenticate users, create accounts, update profiles, verify access permissions	Valid account, login session, user profile
Income management	Amount, transaction date, category, receiving account, notes	Validate data, record income transaction, update account balance	Income list, total income, updated balance
Expense management	Amount, transaction date, category, payment account, payment method	Record expense transaction, deduct balance, compare against budget	Expense list, spending by category, budget alerts
Financial account management	Account name, account type, opening balance, description	Store account configuration, synchronize balance changes from transactions	Account list, current balance, balance history
Budget management	Limit, month, year, scope, warning threshold	Calculate budget usage rate, compare with actual spending	Budget usage, remaining amount, near-limit / exceeded alerts

Subsystem	Input	Process	Output
Debt management	Debt amount, related party, due date, payment	Calculate remaining amount, update debt status, check overdue status	Debt list, debt status, overdue debt alerts
Reporting and dashboard	Income, expenses, financial accounts, budgets, debts, alerts	Aggregate data by day/month/year, group by category, visualize	Overview dashboard, statistics tables, charts, exported files
Backup and scheduled tasks	Transaction data, CronJob configuration, management commands	Generate summaries, check alerts, back up data, run smoke tests	Backup files, periodic summaries, task logs

1.4 Scope of the project

This project focuses on building the core functions of a personal finance management system, including user management, categories, financial accounts, income, expenses, budgets, alerts, debts, sharing groups, and summary reporting. In addition to the business layer, the system is also extended with several technical aspects such as containerization with Docker, deployment with Kubernetes, backup, monitoring, and scheduled tasks in order to demonstrate that the database can operate in a near-realistic environment.

To clearly define the limits of the project, the scope is divided into two groups, *in-scope* and *out-of-scope*, as shown in Table 1.2.

Table 1.2: In-scope and Out-of-scope of the project

Content Group	In-scope	Out-of-scope
Core business functionality	Management of user accounts, personal profiles, income, expenses, categories, financial accounts, budgets, alerts, debts, and reports	Enterprise accounting, financial management for large organizations
Database	MySQL database design with PK, FK, CHECK, UNIQUE, INDEX, VIEW, FUNCTION, PROCEDURE, TRIGGER	Data warehouses, OLAP, large-scale data analytics

Content Group	In-scope	Out-of-scope
Web application	Django backend integration, data entry forms, dashboard, tabular/chart reports	Separate native mobile apps for iOS/Android
Deployment and operations	Docker, Docker Compose, Kubernetes, Helm, CronJob, backup, monitoring, Cloudflare Tunnel for public demo	Multi-region production infrastructure, enterprise fixed domains, complex CDN architecture
Extended features	Debt management, sharing groups, report export, periodic backup	Real-time bank synchronization, AI-based receipt OCR, finance chatbot, ML forecasting, crypto/stock investment analysis

Although the system extends to areas such as Docker, Kubernetes, CronJob, backup, and monitoring, these are only components used to illustrate the practical operability of the system. The main focus of the project remains the design, implementation, and exploitation of a relational database on MySQL in accordance with the orientation of the Database Management Systems course.

1.5 Methodology

The project is implemented through a process consisting of five main steps.

1. Analyze the business domain to identify actors, functions, required data, and important business rules.
2. Design the database from the conceptual data model to the relational schema, including primary keys, foreign keys, and integrity constraints.
3. Implement the database in SQL on MySQL, including tables, views, functions, procedures, triggers, indexes, and sample data.
4. Integrate the database with a Django application to verify data manipulation, querying, and display in a real environment.
5. Deploy, test, and evaluate the system through Docker, Kubernetes, monitoring, backup, and scheduled tasks.

During implementation, the database does not exist independently, but is directly connected to business modules and system outputs such as dashboards, reports, alerts, and scheduled tasks. This relationship is summarized in Table 1.3. The table helps clarify how business requirements are mapped to data structures, processing modules, and corresponding output results.

Table 1.3: Mapping between business requirements, data tables, processing modules, and report/alert outputs

Business Requirement	Related Data Tables	Processing Module	Report / Alert Output
User registration, login, and profile management	users	apps/accounts	Personal profile, login status, access permissions
Recording and managing income	incomes, categories, bank_accounts	apps/income	Total income by day/month/year, overview dashboard
Recording and managing expenses	expenses, categories, bank_accounts	apps/expenses	Expense reports, category charts, budget comparison data
Tracking financial accounts and balances	bank_accounts, incomes, expenses, debt_payments	apps/bank_accounts	Current balance, account movement history
Setting and tracking budgets	budgets, expenses, alerts	apps/budgets, apps/expenses	Budget usage rate, near-threshold / exceeded alerts
Alert management	alerts	apps/alerts	Unread/read alert list, business notifications
Debt management and debt repayment	debts, debt_payments, alerts	apps/debts	Debt status, remaining amount, overdue alerts
Sharing group management	groups, group_members, shared_transactions	apps/sharing	Group list, shared transactions

Business Requirement	Related Data Tables	Processing Module	Report / Alert Output
Dashboard and summary reporting	incomes, expenses, budgets, alerts, debts	apps/dashboard, apps/reports	Dashboard, tabular reports, charts, export
Backup, daily summary, and periodic checks	Aggregated financial data, backup files	apps/reports/management, apps/debts/management, Helm CronJob	Daily summary, backup files, job logs, periodic alerts

This approach helps the system be built through a logical process, from business requirements to the data model, from SQL implementation to application integration and real-world operation.

1.6 Conclusion of the introduction

From the analysis above, it can be seen that the topic of building a personal finance management system is both practically meaningful and aligned with the orientation of the Database Management Systems course. It is a problem that allows relatively comprehensive application of knowledge in requirements analysis, data model design, SQL implementation, integrity control, and integration with a real application. This introductory chapter provides the foundation for the following chapters, which go deeper into problem analysis, database design, system implementation, and evaluation of the achieved results.

2. Problem Analysis

2.1 Problem description and data characteristics

Personal finance management is a problem related to collecting, storing, processing, and analyzing financial transactions that arise in the daily life of an individual or a small group of users. In practice, each person may have many different sources of income such as salary, bonuses, side jobs, or cashback; at the same time, they also incur many expenses in categories such as food, transportation, bills, shopping, education, healthcare, and entertainment. If these income and expense items are not fully recorded, users will have difficulty controlling cash flow and evaluating their spending habits.

The problem does not stop at recording transactions, but also includes deeper management needs such as tracking account balances, checking budget usage, detecting overdue

debts, and viewing financial reports over time. Therefore, the system must support not only data storage but also structured data organization, aggregate querying, business alerts, and clear information presentation.

The data of this problem has four main characteristics. First, the data is clearly relational because every transaction is associated with a user, a category, and potentially a financial account. Second, the data is highly time-based, since most important entities are associated with transaction dates or time points for reporting purposes. Third, financial data requires high accuracy, especially in monetary fields, balances, and aggregated values. Fourth, the data is highly private, so the system must ensure the principle of access control based on data ownership.

2.2 Main actors of the system

The system is analyzed based on three main actors: Guest, User, and Admin. Separating the actors helps clearly define access scope and the functions available to each user group.

Guest refers to a person who has not logged in to the system. This actor is only allowed to access public functions such as viewing the landing page, registering an account, and logging in.

User refers to an authenticated user and is the most important actor in the system. A User can manage their personal profile, financial accounts, income, expenses, budgets, debts, alerts, sharing groups, and reports. One important principle is that each User may only interact with data that belongs to them, except in sharing cases explicitly allowed by the system.

Admin refers to the system administrator. The main role of Admin is to manage operations, users, monitoring, backup, and some system-wide shared data. However, in the context of personal financial data, Admin permissions must also be tightly controlled to ensure privacy.

The relationship between actors, access permissions, and functional scope is summarized in Table 2.1.

Table 2.1: **Actor – Permission – Function Matrix**

Actor	Main Permissions	Allowed Functions
Guest	Public access, unauthenticated	View landing page, register, log in

Actor	Main Permissions	Allowed Functions
User	Manage their own financial data	Manage profile, income, expenses, categories, financial accounts, budgets, alerts, debts, sharing groups, reports, export
Admin	System administration and shared data management	Manage users, manage system categories, monitor logs, backup, monitoring, operational checks

To clarify the access scope and functions of each actor, Figure 2 illustrates the basic permission model among Guest, User, and Admin in the system.

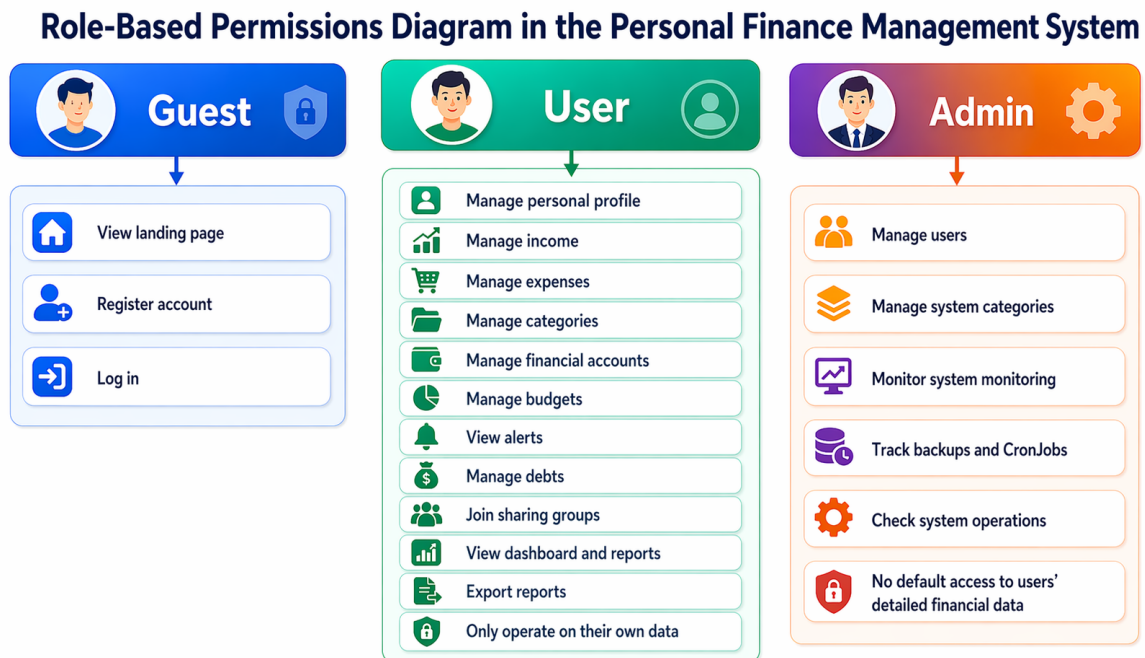


Figure 2: Permission model among actors in the NUFI personal finance management system

From the diagram above, it can be seen that the system is separated according to the principle of clear distinction among public access, personal data manipulation, and operational administration. In particular, User may only operate on data they own, while Admin mainly performs system administration functions.

2.3 Main functions of the system

Based on the analyzed problem, the system functions can be grouped into five main groups.

The first group is **authentication and user management**, including registration, login, profile management, and access control. This is the foundation that ties all financial data to the correct owner.

The second group is **financial transaction management**, including income, expenses, categories, and financial accounts. This is the core of the system, directly reflecting cash inflow, cash outflow, and actual balance changes.

The third group is **financial control**, including budgets, alerts, and debt management. This group of functions helps the system go beyond merely recording data by supporting users in tracking financial risks and making better decisions.

The fourth group is **collaboration and extension**, including sharing groups and shared transactions. These functions support situations where multiple users jointly track some expenses within a small group.

The fifth group is **data exploitation and outputs**, including dashboard, summary reports, and export. This is the part that transforms raw data into meaningful information for users.

2.4 Functional and non-functional requirements

From the functions above, the system must satisfy functional requirements such as account management, income, expenses, categories, financial accounts, budgets, alerts, debts, sharing groups, and reports. In addition, it must satisfy non-functional requirements such as security, data integrity, performance, scalability, recoverability, and usability.

These requirements are especially important for a system related to personal financial data. In terms of security, users should only be able to access their own data. In terms of data integrity, relationships among tables must be controlled through primary keys, foreign keys, and appropriate constraints. In terms of performance, the system must support fast queries at personal-data scale. In terms of operations, the system must provide backup, monitoring, and scheduled tasks to ensure stability.

All functional and non-functional requirements are summarized in Table 2.2.

Table 2.2: **Functional and Non-functional Requirements Matrix**

Requirement Group	Main Content	Objective
Functional requirement	Registration, login, profile management	Authenticate users and personalize data

Requirement Group	Main Content	Objective
Functional requirement	Income and expense management	Fully record personal financial transactions
Functional requirement	Category and financial account management	Standardize data and track balances
Functional requirement	Budget and alert management	Control spending and detect risks early
Functional requirement	Debt management and debt repayment	Track financial obligations over time
Functional requirement	Sharing groups and shared transactions	Support collaborative spending in small groups
Functional requirement	Dashboard, reports, export	Aggregate and visualize data
Non-functional requirement	Security	Ensure that each user can access only their own data
Non-functional requirement	Data integrity	Ensure correct PK, FK, CHECK, UNIQUE, and business states
Non-functional requirement	Performance	Speed up queries through indexes and appropriate data structures
Non-functional requirement	Scalability	Support module expansion and Docker/Kubernetes deployment
Non-functional requirement	Recoverability	Support backup, restore, CronJob, and monitoring

Requirement Group	Main Content	Objective
Non-functional requirement	Usability	Clear interface, easy data entry, and easy-to-read reports

2.5 Conclusion of the problem analysis chapter

Through the analysis process, it can be seen that personal finance management is a highly practical problem with rich data and strict management requirements. The system must not only support the storage of income and expense transactions but also manage balances, budgets, debts, alerts, and reports. Clearly identifying actors, functions, and requirements from the analysis stage is an important foundation for moving to the database design stage in the next chapter.

3. Database Design

The database is the central component of the NUFI system because all data related to users, financial transactions, budgets, alerts, debts, and reports is organized, stored, and exploited on the MySQL platform. In this project, database design is not only intended to support the web application but also to meet the requirements of the Database Management Systems course, including entity analysis, relationship identification, data normalization, primary key and foreign key design, integrity constraints, and extension to SQL objects such as views, functions, procedures, and triggers. Therefore, this chapter focuses on presenting the entire data design process from the conceptual level to the logical implementation level.

3.1 Conceptual data model

At the conceptual level, the NUFI system is built to solve the personal finance management problem for each user. From the real-world problem, the main entities of the system include: users, categories, financial accounts, incomes, expenses, budgets, alerts, debts, debt payments, sharing groups, group members, and shared transactions. Each entity reflects part of the business problem and participates in organizing financial data in a structured way.

In this model, **users** is the central entity because all financial data is tied to a specific owner. From **users**, the system derives business entities such as **categories**,

`bank_accounts`, `incomes`, `expenses`, `budgets`, `alerts`, and `debts`. The `categories` table is used to standardize transaction classification; `bank_accounts` represents sources of money such as bank accounts, e-wallets, or cash; while `incomes` and `expenses` are the two core transaction groups reflecting inflow and outflow of money.

In addition, `budgets` is used to set spending limits by month or by category; `alerts` stores alerts related to budgets or debts; `debts` and `debt_payments` support tracking financial obligations and repayment history. Beyond personal functions, the system also includes the entity group `groups`, `group_members`, and `shared_transactions` to support transaction sharing within groups.

In terms of relationships, the data model of the system mainly includes one-to-many relationships such as between `users` and `incomes`, `expenses`, `bank_accounts`, `budgets`, or `debts`; between `debts` and `debt_payments`; and between `groups` and `group_members`. In addition, some many-to-many relationships are implemented through intermediary tables, the most typical being the relationship between users and sharing groups through `group_members`. This organization helps the data model both reflect the business correctly and facilitate the transformation to the relational schema in the next section.

To clarify the position of the database in the overall system, Figure 3 presents the overall architecture of the NUFI system and the relationship between the web application, the MySQL database, and supporting operational components such as scheduled tasks, monitoring, and reporting.

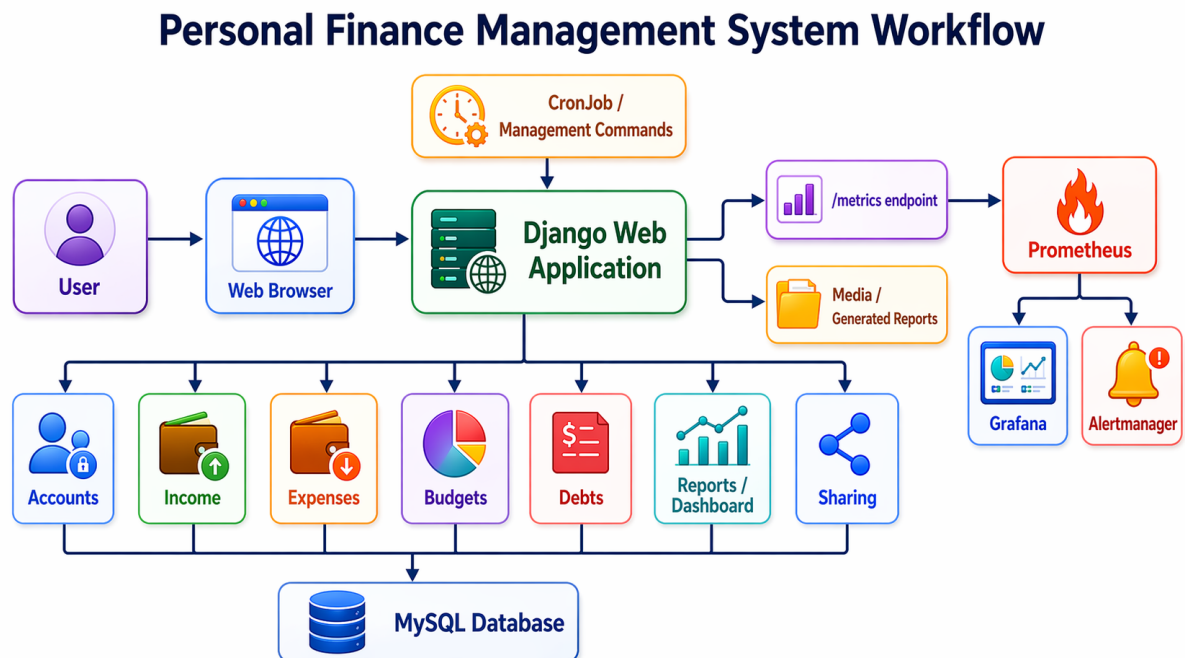


Figure 3: Overall architecture of the NUFI system and the position of the MySQL database in the overall processing flow

From the diagram above, it can be seen that MySQL plays the role of the central storage and data exploitation component of the entire system. Business modules such as account management, income, expenses, budgets, debts, sharing, and reporting all interact directly with the database to record, query, and aggregate information. In addition, the application is extended with operational support components such as CronJob or management commands for scheduled tasks, together with Prometheus, Grafana, and Alertmanager for monitoring and system alerts.

At the data design level, Figure 4 presents a simplified ERD of the system with the main entities and relationships. This diagram helps illustrate the data structure of the system before moving to the stage of building the relational schema and implementing it in MySQL. Through the ERD, it can be seen that **users** is the central entity, from which relationships are formed with categories, financial accounts, incomes, expenses, budgets, alerts, debts, and sharing groups. Correctly identifying entities and relationships at this stage is important because it directly determines the consistency, scalability, and efficiency of data exploitation for the entire system.

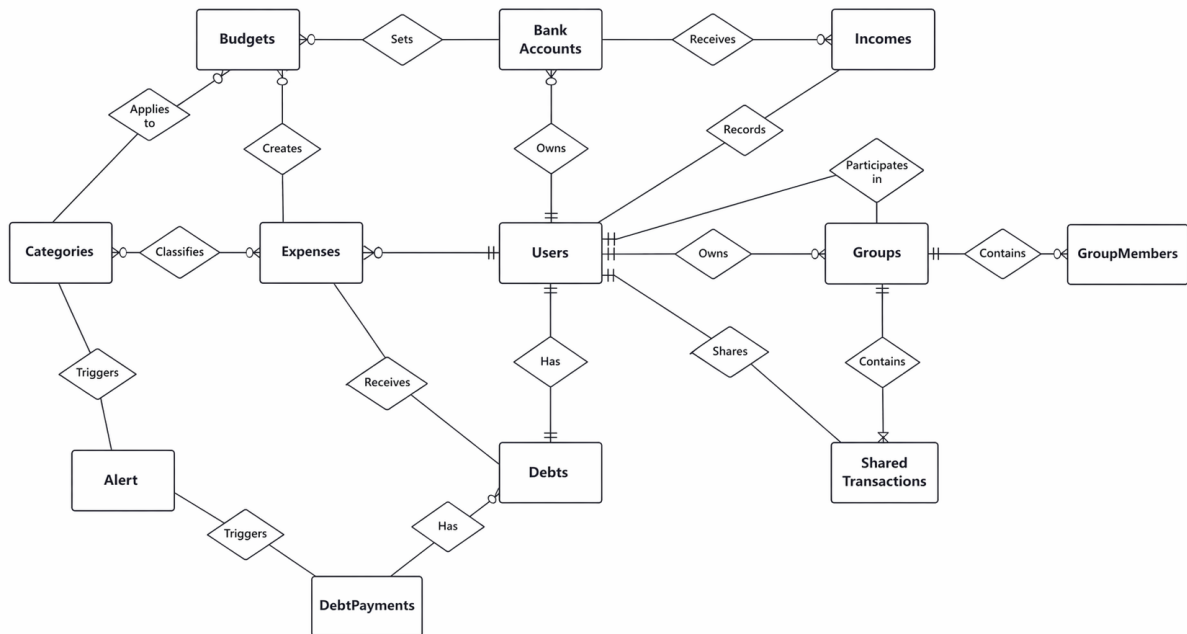


Figure 4: Simplified ERD of the NUFIs personal finance management system

3.2 Transforming the ERD into a relational schema

After completing the conceptual data model and the ERD, the next step is to transform the model into a relational schema for implementation in MySQL. If the ERD mainly describes the problem at the entity and business-relationship level, the relational schema expresses the data at the level of tables, columns, primary keys, foreign keys, and constraints that can be directly implemented in a database management system. This is an

important intermediate step between logical design and physical implementation of the system.

The transformation principles are relatively clear. Each entity in the ERD is mapped to a corresponding table in the database. The main attributes of the entity become data columns, among which one identifying attribute is chosen as the primary key to distinguish each record. For example, the **users** entity is transformed into the **users** table with primary key **user_id**; the **categories** entity is transformed into the **categories** table with primary key **category_id**; similarly, entities such as **bank_accounts**, **incomes**, **expenses**, **budgets**, **alerts**, **debts**, and **debt_payments** are also mapped to independent tables.

For one-to-many relationships, the system uses foreign keys to represent the links between tables. For example, one user can have many income records and many expense records, so the **incomes** and **expenses** tables both contain **user_id** referencing the **users** table. Similarly, one category can be used by many transactions, so **category_id** appears as a foreign key in **incomes**, **expenses**, and **budgets**. A financial account can also be linked to many income, expense, or debt repayment transactions, so **bank_account_id** is used as a foreign key in related tables. This organization helps maintain data consistency and allows querying across multiple business dimensions.

For many-to-many relationships, the system does not store the relationship directly in a list column, but normalizes it into an intermediary table. A typical example is the relationship between users and sharing groups. A user can participate in many groups, and a group can also have many members, so this relationship is separated into the **group_members** table. Likewise, transaction sharing within groups is modeled through the **shared_transactions** table, allowing the system to clearly control which transaction is shared, in which group, and by which user.

In addition to the basic mappings, the transformation to the relational schema also reflects several specific design decisions of the system. For example, the **budgets** table allows **category_id** to be null when the budget is overall, but requires it when the budget is category-based. The **alerts** table can be associated with different types of business objects through columns such as **related_budget_id**, **related_expense_id**, and **related_debt_id**. The **shared_transactions** table is also designed so that one record references either an income transaction or an expense transaction, rather than both at the same time. These details show that the relational schema is not merely a mechanical copy of the ERD, but has been adjusted to suit implementation and business control in MySQL.

Unlike the ERD in the previous section, Figure 5 presents the detailed relational schema of the system after it has been modeled at the level of tables and columns in MySQL. This diagram directly reflects the actual database structure of the project, including table

names, main fields, primary keys, foreign keys, and links among tables.

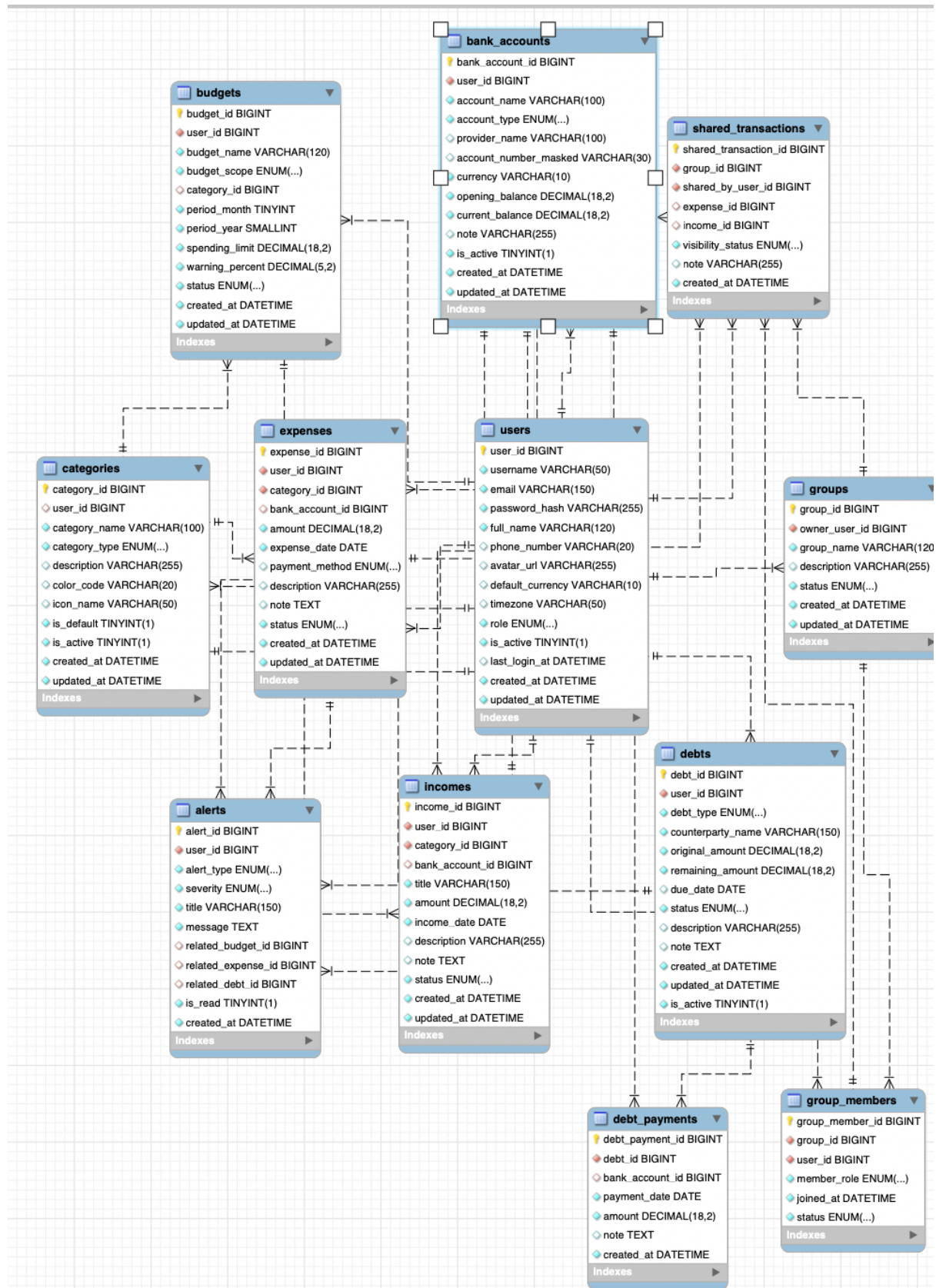


Figure 5: Detailed relational schema of the NUFU personal finance management system

3.3 Data normalization

To ensure that data is not redundant and to avoid anomalies during updates, the NUFI system is designed with normalization up to 3NF according to classical relational database design principles [1, 2]. Normalization helps make data more consistent, reduces duplication, and improves system maintainability.

Table 3.1: **Explanation of data normalization: 1NF / 2NF / 3NF**

Normalization Level	Description	Example in NUFI
1NF	Each column contains only atomic values, not repeated lists	<code>expenses</code> stores only one <code>category_id</code> ; <code>group_members</code> is separated instead of storing a list of members inside <code>groups</code>
2NF	Non-key attributes must depend fully on the primary key	<code>debt_payments</code> is separated from <code>debts</code> because information about each payment does not directly belong to part of a debt key
3NF	Non-key attributes must not depend transitively on other non-key attributes	<code>category_name</code> , <code>icon_name</code> , and <code>color_code</code> are stored only in <code>categories</code> ; transactions only reference <code>category_id</code>

At the 1NF level, each record in the tables is organized in atomic form. For example, the `bank_accounts` table does not store multiple account types in the same column, and the `groups` table does not store the member list as a string.

At the 2NF level, non-key attributes are designed to depend fully on the primary key. For instance, the `debt_payments` table is not merged into `debts`, because one debt can have many payments over time. If merged, data would be duplicated and difficult to update.

At the 3NF level, descriptive information is separated from transaction tables. For example, the category name is stored only in `categories`, while `expenses` stores only `category_id`. This means that when a category name changes, the system does not need to update many old transaction records.

Thanks to normalization, the system avoids three common types of anomalies:

- Insertion anomalies.
- Update anomalies.
- Deletion anomalies.

3.4 Explanation of main tables, primary keys, foreign keys, and integrity constraints

After building the data model and the relational schema, the next step is to clarify the role of each table in the system. Each table in the NUFI database is not only a place to store data, but also directly reflects part of the personal finance management business problem. Viewed as a whole, the system tables can be divided into four main groups: foundational tables for users and classification, financial transaction tables, control and alert tables, and collaboration tables.

First, the **users** table is the central table of the whole system. All financial data must be tied to a specific owner, so **users** plays the role of identifying and separating data among users. This table stores basic information such as username, email, hashed password, role, and active status. Placing **users** at the center helps the system enforce the principle that each user only manipulates data they own.

The **categories** table is used to standardize transaction classification. Instead of storing category names directly in each income or expense record, the system references the **categories** table through foreign keys. This design helps avoid duplicated data, facilitates grouped statistics, and supports user customization of categories according to their needs. Separating categories into a dedicated table also makes it easier to extend with attributes such as color, icon, or active status.

The **bank_accounts** table represents the sources of money managed by users, such as bank accounts, e-wallets, or cash. This is an important table because it allows the system to track actual balances instead of only storing transaction history. When income or expenses occur, transaction records can reference a specific financial account, thereby supporting balance updates and creating a more accurate picture of the current financial state.

The **incomes** and **expenses** tables are the business core of the system. The **incomes** table stores money inflows such as salary, bonuses, cashback, or other income, while **expenses** stores money outflows such as food, transportation, bills, or shopping. Both tables are associated with users, categories, and optionally financial accounts. Separating income and expenses helps the system organize data more clearly, perform statistics more effectively, and build summary reports more conveniently.

The **budgets** table is built to support spending control functionality. Each budget record is usually linked to a user, a defined time period, and optionally a specific category. Through this table, the system can compare actual spending with predefined limits to determine the level of budget usage. This is an important data foundation for alerts and financial decision support.

The **alerts** table stores business alerts generated during system usage, such as nearing

budget limits, exceeding budgets, or overdue debts. Separating alerts into a dedicated table helps the system track alert history, severity, and read/unread status. This design also makes it possible to extend the system with new types of alerts in the future without directly affecting the structure of the main transaction tables.

The **debts** and **debt_payments** tables serve the debt management problem. **debts** stores information about the original debt, related party, principal amount, remaining amount, due date, and current status. Meanwhile, **debt_payments** stores each specific payment for a debt. This separation is necessary because one debt may be repaid multiple times over time. If everything were merged into one table, data would be duplicated and repayment history would be difficult to track.

The group of tables **groups**, **group_members**, and **shared_transactions** is an extension of the system intended to support group-based financial collaboration. The **groups** table stores general information about a sharing group, **group_members** stores the member list and each member's role in the group, and **shared_transactions** stores shared transactions. Thanks to this table group, the system can support cases such as shared expenses in a family, boarding house, or group of friends while still maintaining the boundary between personal data and shared data.

Overall, the main tables of the system are presented in Table 3.2. The table shows that each table has its own business role while also being connected to others to form a complete data model capable of supporting income, expense, budget, debt, alert, and reporting functions in the NUFI system.

Table 3.2: Analysis of primary keys, foreign keys, and integrity constraints

Table	Primary Key	Foreign Keys	Representative Constraints
users	user_id	None	username unique; email unique
categories	category_id	user_id → users	is_default and user_id must be consistent valid category_type
bank_ accounts	bank_account_id	user_id → users	valid opening_balance; valid current_balance
incomes	income_id	user_id → users category_id → categories bank_account_id → bank_accounts	amount > 0; income_date required

Table	Primary Key	Foreign Keys	Representative Constraints
expenses	expense_id	user_id → users category_id → categories bank_account_id → bank_accounts	amount > 0; valid payment_method
budgets	budget_id	user_id → users category_id → categories	period_month from 1 to 12; valid warning_percent
alerts	alert_id	user_id → users budget_id → budgets debt_id → debts	Should reference only the object that caused the alert
debts	debt_id	user_id → users	original_amount > 0; remaining_amount ≥ 0; valid status
debt_payments	payment_id	debt_id → debts bank_account_id → bank_accounts	payment_amount > 0; payment_date required
groups	group_id	owner_user_id → users	group_name must not be empty
group_members	group_member_id	group_id → groups user_id → users	Unique (group_id, user_id)
shared_transactions	shared_transaction_id	group_id → groups shared_by_user_id → users expense_id → expenses income_id → incomes	Only one of expense_id or income_id may have a value

These constraints help the system ensure data consistency. For example, an expense cannot exist without a valid user. A debt payment cannot be created if the original debt does not exist. A group member cannot be added multiple times to the same group.

3.5 Index design and query optimization

In addition to primary keys, foreign keys, and integrity constraints, the NUFi system also uses indexes (INDEX) to improve query performance. In a personal finance management problem, users frequently perform filtering and aggregation by user, time, category, status, or financial account. Without appropriate indexes, these queries would need to scan entire tables, reducing performance as data grows.

The principles of index design in the NUFi system focus on three main groups of queries.

The first group includes queries filtered by data owner, such as retrieving all incomes, expenses, budgets, alerts, or debts of a specific user. The second group includes time-based queries, which support daily, monthly, and yearly reporting, as well as periodic checks such as due debts or alert histories. The third group includes queries combining multiple conditions, such as filtering expenses by user, category, and transaction date to support dashboards and detailed reports.

During design, the system uses both single-column indexes and composite indexes. Single-column indexes are applied to fields commonly used for standalone filtering such as `user_id`, `category_id`, `income_date`, `expense_date`, `due_date`, `status`, or `is_read`. Meanwhile, composite indexes are used for multi-condition query scenarios, for example `(user_id, income_date)` in the `incomes` table, `(user_id, category_id, expense_date)` in the `expenses` table, or `(user_id, is_read, created_at)` in the `alerts` table. These composite indexes are suitable for time-based and status-based aggregate queries in real usage.

In addition to speeding up queries, index design also supports certain data constraints. For example, unique keys such as `uq_users_username`, `uq_users_email`, or `uq_group_members_group` not only prevent duplicate data but also serve as indexes for lookup queries. This shows that indexes in the system are important not only for performance but also for protecting data consistency.

However, the system does not create indexes indiscriminately. Each index is selected based on query frequency and business characteristics because too many indexes can increase write costs when inserting, updating, or deleting records. Therefore, index design in NUFI aims for a balance between read-query speed and data update cost.

Some representative indexes of the system are presented in Table 3.3. The table shows that index creation closely follows real querying needs such as filtering by user, transaction date, status, and condition combinations for reporting.

Table 3.3: **Some representative indexes in the NUFI system**

Table	Representative Indexes	Purpose
users	<code>uq_users_username</code> <code>uq_users_email</code> <code>idx_users_role</code> <code>idx_users_is_active</code>	Account lookup, duplicate-data checking, filtering by role and active status
incomes	<code>idx_incomes_user_id</code> <code>idx_incomes_income_date</code> <code>idx_incomes_user_income_date</code>	Filter income by user, by transaction date, and aggregate income over time

Table	Representative Indexes	Purpose
expenses	idx_expenses_user_id idx_expenses_expense_date idx_expenses_user_expense_date idx_expenses_user_category_expense_date	Support expense queries by user, by date, by category, and for dashboards and reports
budgets	idx_budgets_user_id idx_budgets_period_year_month idx_budgets_user_period_year_month	Query budgets by user and by monthly/yearly period
alerts	idx_alerts_user_id idx_alerts_is_read idx_alerts_created_at idx_alerts_user_is_read_created_at	Display alerts by user, read/unread status, and creation time
debts	idx_debts_user_id idx_debts_status idx_debts_due_date idx_debts_user_status_due_date	Track debts by user, status, and due date
group_members	uq_group_members_group_user idx_group_members_user_id	Prevent duplicate members in the same group and support lookup of a user's groups
shared_transactions	idx_shared_transactions_group_id idx_shared_transactions_shared_by_user_id	Lookup shared transactions by group and by sharing user

Overall, index design in NUFI not only aims to improve query speed, but also helps the system better fit real business operations such as report aggregation, transaction lookup, alert tracking, and overdue debt checks. This is also an important part of demonstrating database design thinking that is both relationally correct and performance-aware.

3.6 Advanced SQL objects in the system

In addition to tables and basic integrity constraints, the NUFI system also uses many advanced SQL objects such as **VIEW**, **FUNCTION**, **PROCEDURE**, and **TRIGGER**. These objects are built not only to satisfy the academic requirements of the Database Management Systems course, but also to organize data logic clearly, improve reusability, and directly support the system's financial business processes. Rather than presenting only general theory, this section describes the specific SQL objects currently used in the system and the purpose of each group.

3.6.1 VIEW

In the NUFI system, **VIEW** is mainly used for reporting, dashboards, and data aggregation by time or category. The existing views help reduce repeated queries in the application layer while also creating a stable and easier-to-use read layer.

Table 3.4: **VIEWS in the NUFI system**

Name of VIEW	Purpose
<code>vw_monthly_expense_summary</code>	Aggregates expenses by user, year, month, and category; supports period-based expense reports and spending-analysis dashboards.
<code>vw_monthly_income_summary</code>	Aggregates income by user, year, month, and category; supports income-source reporting and cash inflow tracking.
<code>vw_budget_usage</code>	Calculates budget usage for each budgets record, including amount used, remaining amount, usage percentage, and status such as <i>normal</i> , <i>warning</i> , and <i>exceeded</i> .
<code>vw_dashboard_snapshot</code>	Creates an overview snapshot for the dashboard per user, including total account balance, current-month income, current-month expenses, net cash flow, unread alerts, and remaining debt.
<code>vw_monthly_financial_summary</code>	Aggregates income, expenses, and net value by month; supports overall financial reporting by period.
<code>vw_category_cashflow_summary</code>	Aggregates cash flow by category, showing transaction count, total income, total expenses, and net value for each category by month.
<code>vw_debt_summary</code>	Aggregates debt status by user, debt type, and status; supports tracking the number of debts, total principal, total remaining amount, and number of overdue debts.

Overall, NUFI's views are built around three main objectives. The first is to support income, expense, and cash-flow reporting over time. The second is to serve the dashboard with aggregated indicators that can be displayed directly in the interface. The third is to support financial control functions such as budget tracking and debt tracking. Thanks to these views, the Django application layer does not need to repeat many complex aggregate

queries.

3.6.2 FUNCTION

If **VIEW** mainly serves read-only data in the form of aggregated tables, then **FUNCTION** in NUFI is used for calculations that return a single value but need to be reused in many different contexts. The existing functions focus on two main business groups: budgets and monthly financial summaries.

Table 3.5: **FUNCTIONs in the NUFI system**

Name of FUNCTION	Purpose
<code>fn_budget_usage_percent</code>	Calculates the percentage of a budgets record that has been used based on actual expenses in the corresponding month.
<code>fn_remaining_budget</code>	Calculates the remaining budget amount of a budgets record.
<code>fn_debt_remaining</code>	Calculates the remaining amount of a debt by subtracting total recorded payments from the original amount.
<code>fn_monthly_income_total</code>	Calculates the total income of a user in a specified month and year.
<code>fn_monthly_expense_total</code>	Calculates the total expenses of a user in a specified month and year.
<code>fn_monthly_net_amount</code>	Calculates the monthly net amount by subtracting total expenses from total income.

These functions help standardize important calculations in the system. Instead of having many queries or application layers compute the same formula separately, the formula is centralized in the database for consistent reuse. This is especially useful in a financial problem, where the same metric, such as monthly total expenses or budget usage percentage, may be used simultaneously in reports, dashboards, alerts, and scheduled tasks.

3.6.3 PROCEDURE

In the NUFI system, **PROCEDURE** is used to model business operations that involve multiple sequential processing steps. These are cases where an action is not simply inserting a

record, but also requires data validation, transaction handling, and related follow-up actions.

Table 3.6: **PROCEDUREs in the NUFI system**

Name of PROCEDURE	Purpose
<code>sp_create_expense</code>	Creates a new expense, validates input data, checks that the financial account belongs to the correct user, inserts the expense transaction, and simultaneously generates a budget alert if spending has exceeded a warning threshold or the budget limit.
<code>sp_create_income</code>	Creates a new income transaction, validates input conditions, checks the financial account, and inserts the income record within a transaction logic.
<code>sp_create_debt_payment</code>	Creates a new debt payment, checks that the debt is still valid, verifies that the payment does not exceed the remaining amount, and validates that the payment account belongs to the owner of the debt.
<code>sp_monthly_summary</code>	Returns a monthly financial summary for a user, including total income, total expenses, net amount, total active account balance, unread alert count, and remaining debt.

It can be seen that NUFI's procedures focus on business operations with high practical value, especially transaction creation and monthly financial summarization. Implementing these processes in procedures increases encapsulation and makes it clear that some important business operations can be executed directly at the database layer rather than only in the application layer.

3.6.4 TRIGGER

The **TRIGGER** group in NUFI is used to validate business constraints at the moment data changes and to automatically update certain derived data such as account balances or debt status. This is the group of objects that most clearly reflects the role of the database in protecting data correctness.

Table 3.7: **TRIGGER** groups in the NUFI system

Trigger Group	Purpose
trg_categories_bi_validate trg_categories_bu_validate	Validates default-category and personal-category rules, ensuring that default categories do not have <code>user_id</code> while personal categories must have <code>user_id</code> .
trg_incomes_bi_validate trg_incomes_bu_validate	Checks whether the income category is valid, active, and whether the financial account belongs to the correct user before inserting or updating income.
trg_incomes_ai_update_balance trg_incomes_au_update_balance	Automatically increases account balance when active income is inserted and readjusts the balance when an income record is updated.
trg_expenses_bi_validate trg_expenses_bu_validate	Checks the expense category and financial account before inserting or updating an expense, ensuring that the expense is linked only to a valid category and an account owned by the correct user.
trg_expenses_ai_update_balance trg_expenses_au_update_balance	Automatically deducts account balance when an expense is inserted and readjusts the balance when an expense is updated.
trg_budgets_bi_validate trg_budgets_bu_validate	Validates overall-budget and category-budget logic, checks budget category validity, and prevents duplicate active budgets for the same user in the same period.
trg_alerts_bi_validate trg_alerts_bu_validate	Ensures that an alert references at most one related object; also requires budget alerts to have <code>related_budget_id</code> , overdue-debt alerts to have <code>related_debt_id</code> , and system alerts to have no business object attached.

Trigger Group	Purpose
trg_shared_transactions_bi_validate trg_shared_transactions_bu_validate	Validates that a shared transaction references only one type of transaction (income or expense), that the sharing user is an active group member, and that the shared transaction really belongs to that user.
trg_debt_payments_bi_validate trg_debt_payments_bu_validate	Validates that a debt payment applies only to a valid debt and that the payment account belongs to the owner of that debt.
trg_debt_payments_ai_update_debt trg_debt_payments_au_update_debt	Automatically updates remaining_amount and status of a debt after a repayment is inserted or updated; statuses may change to <i>pending</i> , <i>partially_paid</i> , <i>overdue</i> , or <i>paid</i> .

Overall, NUFI's triggers are used for two groups of objectives. The first group is to validate data before it is written to the database, such as validating categories, financial accounts, budgets, alerts, and shared transactions. The second group is to automatically update derived data after changes occur, most notably account balances and debt status. This design helps reduce the risk of data inconsistency when there are multiple data-writing paths in the system.

3.6.5 Overall assessment

It can be seen that each group of SQL objects in NUFI performs a different role. **VIEW** mainly supports read-oriented data exploitation and reporting. **FUNCTION** supports single-value calculations that are reused many times. **PROCEDURE** supports business operations consisting of multiple steps. **TRIGGER** is responsible for data validation and automatic updates at the database level. The combination of these objects shows that the system does not use MySQL merely as a place to store tables, but also exploits advanced DBMS capabilities to support the personal finance management problem in a more rigorous and practical way.

3.7 Separation of logic between Database and Django

An important design decision is not to push all business logic into the database. The NUFI system chooses to separate responsibilities between MySQL and Django in order to both ensure data integrity and make the application easier to maintain, test, and extend.

Table 3.8: Separation of logic between Database and Django

Logic better placed in the Database	Logic better placed in Django
PK, FK, CHECK, UNIQUE, NOT NULL	Access control based on the logged-in user
Views and functions for data aggregation	Interface navigation and user-friendly error display
Some triggers for low-level data synchronization	Generating alerts immediately after user actions
Relational constraints among tables	Avatar upload, file export, dashboard rendering
Protecting correctness and consistency of data	Management commands, CronJob, monitoring integration

The database should be responsible for aspects directly related to data consistency, such as keys, constraints, relationships, and part of the read/aggregate calculation logic. This is the last line of defense that keeps data correct even when there are multiple writing paths into the system.

In contrast, Django should handle application-level logic such as permissions, user experience, input forms, error messages, interface navigation, alert generation during interaction flows, and integrations with backup, monitoring, or dashboards. If everything is pushed into triggers or procedures, the system becomes harder to maintain and harder to explain at the web layer. If everything is pushed into Django, the data becomes more vulnerable to integrity issues. Therefore, the separation above is the most appropriate for NUFI.

3.8 Conclusion

Through the database design process, it can be seen that the NUFI system has built a relatively complete relational data model for the personal finance management problem. This model correctly reflects the main business entities, clearly defines relationships among tables, applies normalization up to 3NF, uses keys and integrity constraints appropriately, and extends to advanced SQL components such as views, functions, procedures, and triggers.

Thanks to this well-structured data foundation, the system can effectively support income management, expense management, budgets, alerts, debts, sharing groups, and reporting. This is also the important basis for the next chapter, where the entire design is implemented through SQL scripts on MySQL.

4. Database Implementation Using SQL

If Chapter 3 focused on the data model and design decisions, Chapter 4 presents how that model is materialized through SQL scripts on MySQL. The focus of this chapter is no longer to re-explain entities, relationships, or design rationale, but to describe the structure of SQL files, execution order, DDL organization, implementation of SQL objects, and the use of sample data for testing. In other words, Chapter 3 answers how the system is designed, while Chapter 4 answers how that design is implemented in a real database.

4.1 Introduction to the database management system used

The NUFI system uses MySQL 8.x as the main database management system [3]. Choosing MySQL is suitable for the nature of the project because it is a popular relational DBMS that provides good support for system components such as tables, primary keys, foreign keys, indexes, views, functions, procedures, triggers, and transactions. MySQL also integrates conveniently with Django, is easy to deploy in local, Docker, and Kubernetes environments, and is powerful enough to model a personal finance management system that can operate in practice.

In the NUFI project, the main database is named **pfms**. All SQL scripts are organized in the **sql/** directory and separated clearly by function, including files for table structure definition, sample data creation, views, functions, procedures, triggers, and indexes. This organization makes installation, testing, and project presentation clearer, while also making database re-creation more convenient.

4.2 Execution order of SQL files

To ensure that the database is installed correctly and avoids dependency errors among objects, the SQL files in the project must be executed in a proper order. In principle, the foundational structure must be created first, followed by calculation, validation, and aggregation objects, and finally sample data. In the NUFI system, the execution order is chosen to ensure that tables, advanced SQL objects, and illustrative data all work correctly immediately after installation.

First, **schema.sql** is executed to create the **pfms** database and all main tables of the system. Then the system successively deploys **FUNCTION**, **TRIGGER**, **VIEW**, and **PROCEDURE**. Finally, **indexes.sql** is used to deploy additional indexes and re-apply them explicitly, while **sample_data.sql** is used to load sample data for testing and demonstration.

To illustrate the database installation order of the project, Figure 6 presents the installation flow from schema to sample data.

Database Initialization Flow for Demo / Test / App Integration

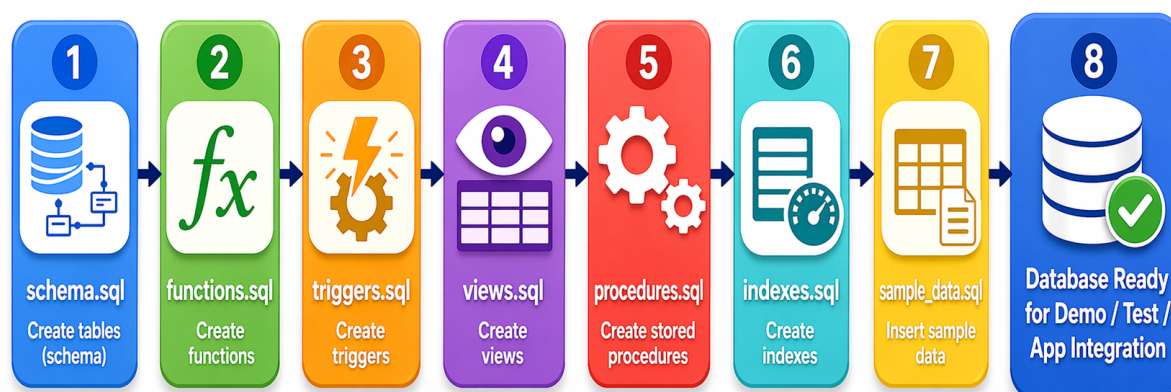


Figure 6: Database installation flow from schema to sample data

Table 4.1 presents the execution order of SQL files in the project and the role of each file.

Table 4.1: Execution order of SQL files in the project

Step	SQL File	Object Group	Role
1	sql/schema.sql	Foundational structure	Creates the pfms database, all tables, primary keys, foreign keys, unique keys, and some basic indexes.
2	sql/functions.sql	Calculation functions	Creates functions used for budget, debt, and monthly financial summary calculations.
3	sql/triggers.sql	Validation and automatic-update triggers	Creates triggers to validate data and automatically update account balances, debt status, or related business rules.
4	sql/views.sql	Reporting views	Creates views for dashboards, summary reports, and analytical data queries.

Step	SQL File	Object Group	Role
5	sql/procedures.sql	Business procedures	Creates stored procedures for multi-step operations such as creating income, expenses, debt payments, and monthly summaries.
6	sql/indexes.sql	Additional indexes	Creates or re-applies indexes explicitly and idempotently through the supporting procedure <code>sp_add_index_if_not_exists</code> .
7	sql/sample_data.sql	Sample data	Loads illustrative data for users, categories, accounts, transactions, budgets, debts, alerts, sharing groups, and shared transactions.

4.3 Implementing table structure with DDL

The implementation of the system's data structure is mainly concentrated in the `sql/schema.sql` file. This file not only creates tables, but also sets up the database, chooses the `utf8mb4` character set, `utf8mb4_unicode_ci` collation, and the InnoDB engine. This shows that implementation goes beyond creating isolated tables and has been organized as a complete script for rebuilding the database.

One notable point is that `schema.sql` performs `DROP DATABASE IF EXISTS` and `CREATE DATABASE` before creating tables. This approach suits the goal of resetting the installation environment for demo, testing, and project submission purposes, because it allows the entire database to be recreated from scratch with a consistent structure. After creating the database, the tables are deployed in a logical order to ensure that foreign keys always reference tables that already exist.

In terms of execution, foundational tables such as `users`, `categories`, and `bank_accounts` are created first. Then come transaction tables such as `incomes` and `expenses`, followed by business-control tables such as `budgets`, `debts`, `alerts`, and `debt_payments`. Finally come collaboration-support tables such as `groups`, `group_members`, and `shared_transactions`. This ordering follows data dependencies among tables and limits foreign-key errors during deployment.

Table 4.2 lists the main tables of the system and the corresponding DDL script.

Table 4.2: List of tables and corresponding DDL script

Table	DDL Script	Implementation Notes
<code>users</code>	<code>sql/schema.sql</code>	Root table of the system, created first so that other tables can reference it via foreign keys.
<code>categories</code>	<code>sql/schema.sql</code>	Contains default and personal categories; has a foreign key to <code>users</code> .
<code>bank_accounts</code>	<code>sql/schema.sql</code>	Stores bank accounts, e-wallets, and cash; has a foreign key to <code>users</code> .
<code>incomes</code>	<code>sql/schema.sql</code>	Stores income; references <code>users</code> , <code>categories</code> , and <code>bank_accounts</code> .
<code>expenses</code>	<code>sql/schema.sql</code>	Stores expenses; references <code>users</code> , <code>categories</code> , and <code>bank_accounts</code> .
<code>budgets</code>	<code>sql/schema.sql</code>	Stores overall or category-based budgets; references <code>users</code> and <code>categories</code> .
<code>debts</code>	<code>sql/schema.sql</code>	Stores debts and receivables; references <code>users</code> .
<code>alerts</code>	<code>sql/schema.sql</code>	Stores business alerts; references <code>users</code> and related objects such as <code>budgets</code> , <code>expenses</code> , and <code>debts</code> .
<code>debt_payments</code>	<code>sql/schema.sql</code>	Stores each debt repayment; references <code>debts</code> and <code>bank_accounts</code> .
<code>groups</code>	<code>sql/schema.sql</code>	Stores sharing-group information; references <code>users</code> through <code>owner_user_id</code> .
<code>group_members</code>	<code>sql/schema.sql</code>	Intermediary table between users and groups; contains a unique key to prevent duplicate members.
<code>shared_transactions</code>	<code>sql/schema.sql</code>	Stores transactions shared within groups; references <code>groups</code> , <code>users</code> , <code>incomes</code> , and <code>expenses</code> .

From an implementation perspective, `schema.sql` is not just a list of `CREATE TABLE` statements, but the physical realization of the entire data model built in Chapter 3. Thanks to this, the NUFi database can be recreated with a consistent structure and be ready for the next steps of advanced logic implementation.

4.4 Implementing data constraints and validation rules

The role of primary keys, foreign keys, and integrity constraints was explained in Chapter 3. In Chapter 4, the focus shifts to how these constraints are implemented in actual SQL scripts. Specifically, `schema.sql` directly declares components such as `PRIMARY KEY`, `FOREIGN KEY`, `UNIQUE`, `NOT NULL`, `DEFAULT`, and `ENUM` for each table. This is the first layer of control that prevents invalid data at the time of insert or update.

For example, the `users` table defines uniqueness for `username` and `email`; transaction tables such as `incomes`, `expenses`, and `debt_payments` impose strict requirements on monetary fields and transaction dates; tables such as `budgets`, `debts`, `alerts`, and `group_members` use `ENUM` to restrict the set of valid values for status or business type.

However, not all business rules are convenient to express entirely through static DDL. Some rules depend simultaneously on multiple columns or multiple tables, such as the relationship between `is_default` and `user_id` in the `categories` table, the relationship between `budget_scope` and `category_id` in the `budgets` table, or the rule that one alert may reference at most one related object in the `alerts` table. Therefore, at the implementation level, the system combines DDL constraints with validation logic in triggers or procedures to ensure data correctness.

The important point is that the NUFI database does not merely create tables, but is also implemented with real data-protection layers. This allows the system not to rely entirely on the application layer for validation, but to take advantage of the data-control role of the DBMS.

4.5 Implementing indexes

The basis for index selection and the role of indexes in query performance were explained in Chapter 3. In this section, the discussion shifts to the implementation perspective. Specifically, many of the system's basic indexes are already declared directly in `sql/schema.sql`, especially unique keys, indexes on `user_id`, `category_id`, `income_date`, `expense_date`, `due_date`, and some composite indexes for dashboards or reports.

In addition, the project includes `sql/indexes.sql` to implement indexes in a more explicit way. This file defines the procedure `sp_add_index_if_not_exists` to create indexes idempotently, meaning that the script can be re-run without duplicating indexes that already exist. This is especially useful during environment resets, testing, and repeated demos. Therefore, the index implementation in the project not only serves query optimization but also supports the reproducibility of database deployment.

4.6 Implementing VIEW, FUNCTION, PROCEDURE, and TRIGGER

The business roles of each component in VIEW, FUNCTION, PROCEDURE, and TRIGGER were already explained in Chapter 3. Therefore, in Chapter 4, the focus is no longer on re-explaining the meaning of each object group, but on describing how these objects are organized and implemented in the project's SQL files.

Specifically, `sql/views.sql` contains the views used for dashboards and summary reports; `sql/functions.sql` contains calculation functions for budgets, debts, and monthly financial summaries; `sql/procedures.sql` contains stored procedures for multi-step business operations; and `sql/triggers.sql` contains triggers used to validate input data and automatically update derived values such as account balances or debt status. In addition, `sql/indexes.sql` is separated to deploy additional indexes and re-apply them more explicitly.

Thus, from the implementation perspective, the advanced SQL objects of the system are not placed randomly, but are divided into files by function. This organization makes deployment, testing, and database maintenance more convenient, while also clarifying the relationship between data structure and logical components implemented in MySQL.

Table 4.3 is intended only as a quick summary, illustrating the main SQL object groups currently present in the system and their implementation-level roles.

Table 4.3: **INDEX, VIEW, FUNCTION, PROCEDURE, TRIGGER list and roles**

Group	Representative Objects	Role
INDEX	<code>uq_users_username</code> , <code>uq_users_email</code> , <code>idx_incomes_user_income_date</code> , <code>idx_expenses_user_category_expense_date</code> , <code>idx_budgets_user_period_year_month</code> , <code>idx_alerts_user_is_read_created_at</code> , <code>idx_debts_user_status_due_date</code>	Support query optimization by user, time, category, and status.
VIEW	<code>vw_monthly_expense_summary</code> , <code>vw_monthly_income_summary</code> , <code>vw_budget_usage</code> , <code>vw_dashboard_snapshot</code> , <code>vw_monthly_financial_summary</code> , <code>vw_category_cashflow_summary</code> , <code>vw_debt_summary</code>	Support aggregate queries for dashboards, reporting, budget monitoring, and debt tracking.

Group	Representative Objects	Role
FUNCTION	fn_budget_usage_percent, fn_remaining_budget, fn_debt_remaining, fn_monthly_income_total, fn_monthly_expense_total, fn_monthly_net_amount	Standardize repeated calculations during data exploitation.
PROCEDURE	sp_create_expense, sp_create_income, sp_create_debt_payment, sp_monthly_summary	Support multi-step operations and some processing flows using transactions.
TRIGGER	trg_categories_*, trg_incomes_*, trg_expenses_*, trg_budgets_*, trg_alerts_*, trg_shared_transactions_*, trg_debt_payments_*	Validate data and automatically update some derived values at the database level.

Overall, the implementation of advanced SQL objects shows that NUFI's database is not built only at the level of basic tables, but is also organized into components that support querying, calculation, validation, and automatic updates. This contributes to making the system easier to exploit during application integration and post-deployment testing.

4.7 Sample data and its testing significance

After the entire structure and SQL logic are implemented, the `sql/sample_data.sql` file is used to load sample data. This data is not only meant to populate tables for demo purposes, but is also built according to specific business scenarios such as having admin and regular users, default categories, multiple financial accounts, income and expense transactions, monthly budgets, debts, debt repayments, budget alerts and overdue debt alerts, as well as sharing groups and shared transactions.

From an implementation perspective, sample data helps verify that tables, keys, indexes, functions, procedures, triggers, and views can all operate immediately after installation. For example, the dashboard can immediately display balances and net cash flow, aggregate views can produce meaningful results, triggers that update balances and debt status can be observed directly, and budget or overdue-debt alerts already have prepared data for demonstration. This makes testing, demo, and evaluation more substantive rather than stopping at the level of creating an empty structure.

4.8 Conclusion

Through the database implementation process using SQL, it can be seen that the NUFI system has been implemented relatively comprehensively on MySQL 8.x, from foundational DDL to components that support data exploitation and control. The database includes not only tables and keys, but also views, functions, procedures, triggers, indexes, and sample data to support reporting, business validation, and realistic operational simulation.

If Chapter 3 showed how the system was designed, then Chapter 4 proves that the entire design has been materialized into executable SQL scripts that can rebuild the database and directly support the NUFI application in a real environment. This serves as the basis for moving to the following topics such as business rules, Django integration, and system evaluation.

5. Business Rules and Their Implementation in the Database

After completing the design and database implementation, the next step is to clarify the core business rules of the system and how these rules are implemented in practice. For a personal finance management system such as NUFI, data must not only be structurally correct but also business-wise correct. For example, one user must not be able to access another user's data, an expense must change the account balance, an overdue debt must transition to the appropriate status, and a shared transaction must only be created by a valid member of a group.

In the NUFI system, business rules are not concentrated entirely in one layer. One part is enforced directly in the database through keys, constraints, triggers, and procedures to protect data consistency. The remaining part is handled in the Django backend to control access according to the logged-in user, coordinate operation flows, display appropriate notifications, and execute tasks that depend on application context. This separation helps keep the system both rigorous at the data layer and flexible at the application layer.

5.1 Main business rules of the system

NUFI's business rules can be divided into five main groups. The first group concerns data ownership, ensuring that each user may only manipulate records that belong to them or to groups in which they are a valid member. The second group concerns income, expenses, and account balances, in which each transaction must be associated with a valid financial account and change the balance in the correct direction. The third group concerns budgets and alerts, ensuring that budgets are created with the correct structure

and that the system can detect when spending approaches or exceeds limits. The fourth group concerns debts and overdue status, ensuring that payments do not exceed the remaining amount and that debt status is updated correctly according to actual progress. The fifth group concerns shared group data, ensuring that sharing occurs only within valid groups and under the correct data ownership.

Table 5.1 summarizes the main business rules of the system and how they are reflected in implementation.

Table 5.1: **List of main business rules of the system**

Rule Group	Main Content	Key Implementation Approach
Data ownership	Users may only view, create, update, or delete data that belongs to them; group data may only be accessed by the group owner or valid members	Foreign keys, queries filtered by <code>user_id</code> , <code>request.user</code> checks, group membership checks
Income, expenses, and account balances	Transactions must be linked to valid accounts; income increases balances, expenses decrease balances	Triggers that update <code>current_balance</code> , transaction-creation procedures, backend transactions
Budgets and alerts	Overall budgets and category budgets must follow the correct structure; alerts must be attached to the correct related object	Triggers validating <code>budget_scope</code> , triggers validating <code>alerts</code> , alert-generation logic in procedures and Django
Debts and overdue status	Debt payments must not exceed the remaining amount; debt status must reflect the actual current condition	Triggers updating <code>remaining_amount</code> , debt-payment procedures, service/command for overdue checks
Shared group data	Shared transactions may only be created by active members and must belong to the user sharing them	Triggers validating <code>shared_transactions</code> , forms and views restricting transactions to the current user

5.2 Separation of enforcement between SQL and Django

In the NUFI system, not every business rule is suitable to be placed entirely in the database. Rules that represent immutable data invariants and can be checked immediately when data is written are pushed down into SQL to ensure integrity even when data enters from multiple paths. In contrast, rules that depend on the currently logged-in user, the operating interface, user feedback, or scheduled tasks are usually placed in the Django backend. This helps the system avoid two extremes: depending too much on the application and making data easy to corrupt, or putting too much logic into the database and making processing flows hard to trace and maintain.

Specifically, triggers in MySQL currently enforce many important validations such as: default categories and personal categories must have the correct `user_id`; income and expense transactions must be linked to the correct type of category and an account owned by the correct user; overall budgets must not be linked to a category while category budgets must require one; an alert may reference at most one related object; a shared transaction may reference only one income or expense; and a debt payment may only apply to a valid debt and valid payment account. In addition, triggers automatically update account balances and remaining debt amounts after data changes.

On the Django side, the backend handles aspects tied to real usage context. Views always filter data according to the current user profile, for example retrieving only **Expense**, **Income**, **Budget**, **Debt**, or **BankAccount** records owned by the correct user. In the sharing module, Django further checks who is the group owner, who is an active member, who has been invited, and who is allowed to accept invitations. In addition, the backend is responsible for invoking transactions in the correct flow, generating notifications for users, creating alerts during web interaction flows, and running scheduled management commands such as budget checks or overdue debt checks.

Table 5.2 presents more clearly the relationship between each business rule and the corresponding enforcement layer.

Table 5.2: Mapping between business rules and enforcement layer (SQL / Django)

Business Rule	Enforced in SQL	Enforced in Django	Rationale for Separation
Users may only manipulate their own data	FKs by <code>user_id</code> ; some triggers validate ownership relations	<code>request.user</code> , <code>get_object_or_404(...,</code> <code>user=profile)</code>	SQL preserves data linkage; only Django knows who is currently logged in to block unauthorized access

Business Rule	Enforced in SQL	Enforced in Django	Rationale for Separation
Income / expenses must be linked to valid accounts and categories	Triggers validate category type, category status, and account ownership	Form queryset filtered by user, input validation	Conditions that can be checked immediately during data writes should be placed in SQL; the backend mainly supports the input flow
Income increases balances, expenses decrease balances	Triggers update current_balance	Helper adjust_balance used as a fallback layer	SQL guarantees consistency at the data level; Django coordinates compatibility when the environment does not have triggers
Overall / category budgets must have the correct structure	Triggers validate budget_scope , valid category, and prevent duplicate budgets	Forms validate category selection and normalize input data	SQL blocks invalid data; Django helps users enter correct data from the form stage
Alerts must be linked to the correct related object	Triggers validate related_budget_id , related_debt_id , and the number of related objects	Views / services create alerts in the correct business context	SQL validates alert structure; Django decides when and how alerts are created in the application flow
Debt payment must not exceed the remaining amount	Procedure sp_create_debt_payment ; triggers validate the debt and the payment account	Forms limit payment amount and check active accounts	Both layers are needed: the backend shows errors early to users, while SQL blocks incorrect writes
Debt status and remaining amount must be updated correctly	Triggers update remaining_amount and status	Service sync_debt_status_and_alert , command debt_overdue_check	SQL handles changes immediately after payment; Django is better suited for logic depending on current dates and scheduled tasks
Shared transactions must be created by valid members	Triggers validate exactly one of expense_id / income_id , and check active membership	Views validate membership, forms only allow transactions of the current user	SQL protects data relationships; Django controls the interaction flow based on the logged-in user

5.3 Flow for creating an expense and generating an alert

A typical example of coordination between the database and the backend is the flow of creating a new expense. When a user submits the expense creation form, Django receives the data, associates the expense with the current user, and saves the record within a transaction. After that, the account balance is updated accordingly. If the system is running on MySQL with the full set of triggers, the balance update is performed directly in the database; otherwise, the backend uses a fallback layer to ensure the balance is still adjusted correctly. Next, the system compares the newly created expense against active budgets in the month to determine whether a warning threshold has been reached or the limit has been exceeded. If so, a corresponding alert record is created.

To illustrate this flow more clearly, Figure 7 describes the process from creating an expense to updating the balance, checking the budget, and generating an alert.

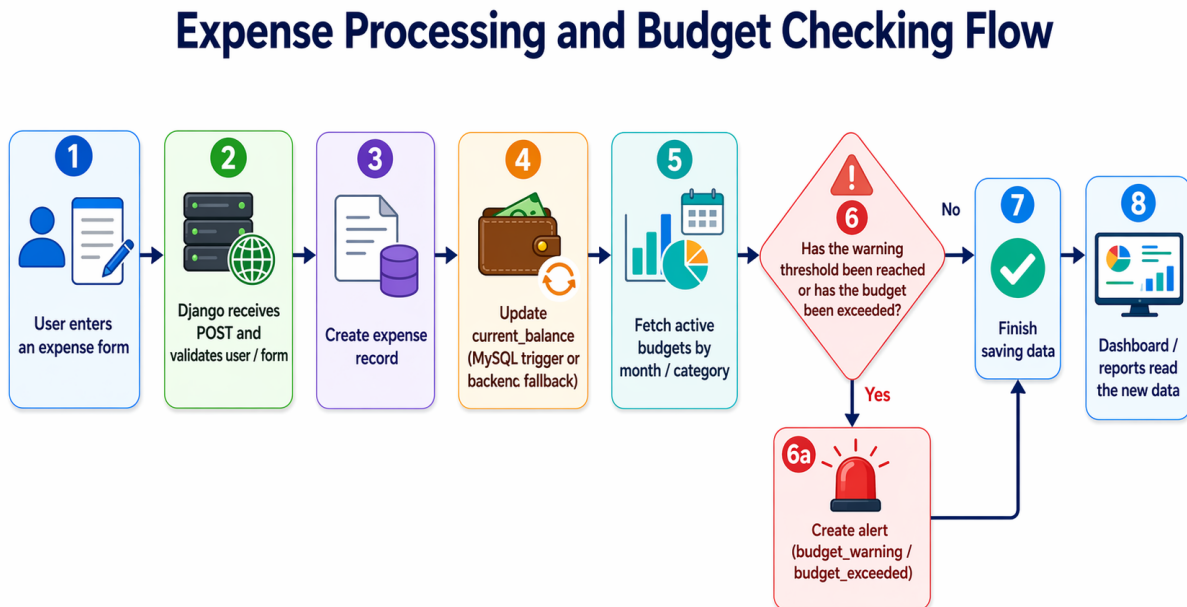


Figure 7: Business flow for creating an expense, updating balance, checking budget, and generating an alert

From the diagram above, it can be seen that a complete business rule often does not reside entirely in one layer. The data write and balance update can be strongly protected by SQL, but deciding which alert should be generated, how its content should be displayed, or whether periodic monthly checks should run is better suited to the Django backend. This is also why the system chooses a coordinated two-layer approach instead of pushing all processing into one side.

5.4 Why not put all logic into triggers and procedures?

In theory, many business rules could be pushed down into triggers or stored procedures. However, if all logic is concentrated in the database, the system would face several significant limitations. First, the database does not know the context of the logged-in session and is not suitable for handling permissions at the level of the current interface user. Decisions such as who may open a page, who may edit a record, who is the group owner, or who is merely a pending member still need to be handled in Django.

Second, triggers and procedures are very suitable for short, local rules directly tied to data-write events, but are not always appropriate for longer business flows or time-dependent logic. For example, determining whether a debt is overdue based on the current date and generating periodic alerts is clearer, more explicit, and easier to test when placed in a Django service or management command rather than hidden entirely in triggers.

Third, if too much logic is packed into triggers or procedures, business flows become difficult to trace. When a web action causes many hidden effects at the database level, debugging, testing, and maintenance become more complex, especially when the system must also integrate with interfaces, reports, dashboards, and background tasks. On the other hand, if all logic is placed only in Django without SQL-level protection, data becomes easier to corrupt when there are multiple data-write paths.

Therefore, the most appropriate approach for NUFI is a separation of responsibilities. SQL is responsible for protecting data invariants, ownership relationships, directly derived updates, and part of the business operations that can be clearly encapsulated. Django is responsible for permissions, coordinating interaction flows, user feedback, running scheduled tasks, and handling parts that depend on application context. This combination both aligns with the spirit of the DBMS course and fits a practical system.

5.5 Conclusion

Through the analysis of NUFI's business rules, it can be seen that the database is not merely a storage layer but also directly participates in protecting data correctness. Rules such as record ownership, account balance updates, budget structure, debt repayment, and shared transactions are implemented through the cooperation of SQL and the Django backend.

This division helps the system achieve two important objectives simultaneously: data remains consistent at the database level, while the application-level processing flow remains clear, extensible, and aligned with user experience. This is also the foundation for moving to the next section on integrating the database with the application and illustrating the completed features of the system.

6. Database Integration with the Application and Feature Illustration

After the database has been designed and implemented in SQL, the next step is to integrate that data structure with the web application to form a system that can operate in practice. In the NUFI project, the Django backend acts as the intermediary layer between the user interface and MySQL. Users interact with forms on the web interface, Django receives the request, validates the data, maps it to corresponding models, writes or reads data from MySQL, and then returns the results to the interface in the form of lists, dashboards, alerts, or reports.

This section does not go deeply into web technology as if it were a pure software-development project, but instead focuses on illustrating how the database is used in the application and how the main functions of the system exploit the data structures built in previous chapters. In other words, this section serves as evidence that the DBMS design of the system does not stop at theory, but has been integrated into a usable application.

6.1 Django backend and mapping to MySQL

The NUFI system uses Django as the main backend, with Django ORM, model metadata, and form handling acting as the intermediary layer between the interface and MySQL [4, 5, 6]. An important point in the integration process is that the Django models are mapped relatively directly to the tables created in MySQL. Instead of using default table names automatically generated by Django, many models explicitly declare `db_table` to stay consistent with the system's relational schema. For example, the `UserProfile` model maps to the `users` table, `BankAccount` maps to `bank_accounts`, `Income` maps to `incomes`, `Expense` maps to `expenses`, `Budget` maps to `budgets`, `Alert` maps to `alerts`, `Debt` maps to `debts`, `DebtPayment` maps to `debt_payments`, while the sharing group maps to `groups`, `group_members`, and `shared_transactions`.

This mapping provides two clear benefits. First, the backend can manipulate data conveniently through Django ORM while preserving the data structure already designed and implemented at the SQL layer. Second, it helps the system maintain consistency between the modeling described in the report and the actual implementation in the source code. As a result, every user action in the interface can be traced back to the corresponding table in MySQL.

Table 6.1 presents the mapping between the main Django modules and the data tables they use.

Table 6.1: Mapping between Django modules and the data tables they use

Django Module	Data Tables Used	Integration Role
accounts	users	Manage user profiles, identify data owners, and serve as the linkage point for other modules.
categories	categories	Manage income and expense categories; provide classification data for forms and reports.
bank_accounts	bank_accounts, incomes, expenses, debt_payments	Manage financial accounts, current balances, and balance history based on transactions or debt payments.
income	incomes, categories, bank_accounts	Record income, classify it by category, and associate it with the receiving financial account.
expenses	expenses, categories, bank_accounts, budgets, alerts	Record expenses, update balances, check related budgets, and generate alerts when necessary.
budgets	budgets, categories, expenses, alerts	Manage monthly or category-based budgets and track usage based on actual expense data.
debts	debts, debt_payments, bank_accounts, alerts	Manage debts, payment records, remaining status, and overdue alerts.
sharing	groups, group_members, shared_transactions, incomes, expenses	Support sharing groups, member management, and sharing income or expense transactions within groups.
dashboard	incomes, expenses, bank_accounts, budgets, alerts	Aggregate current-month data to display an overview dashboard, balances, cash flow, and alerts.
reports	incomes, expenses, bank_accounts, budgets	Generate summary reports by month, year, category, and trend; support Excel/PDF export.

6.2 Data flow from input forms to MySQL

At the user-interaction level, data usually follows a fairly consistent flow. Users enter information into web forms, Django receives the data from the request, validates it through forms and models, attaches the record to the current user, and then uses the ORM to write it into MySQL. After the save operation is completed, the system reads the updated data from the database to display lists, dashboards, or alerts.

Figure 8 illustrates the overall data flow from input forms to the MySQL database.

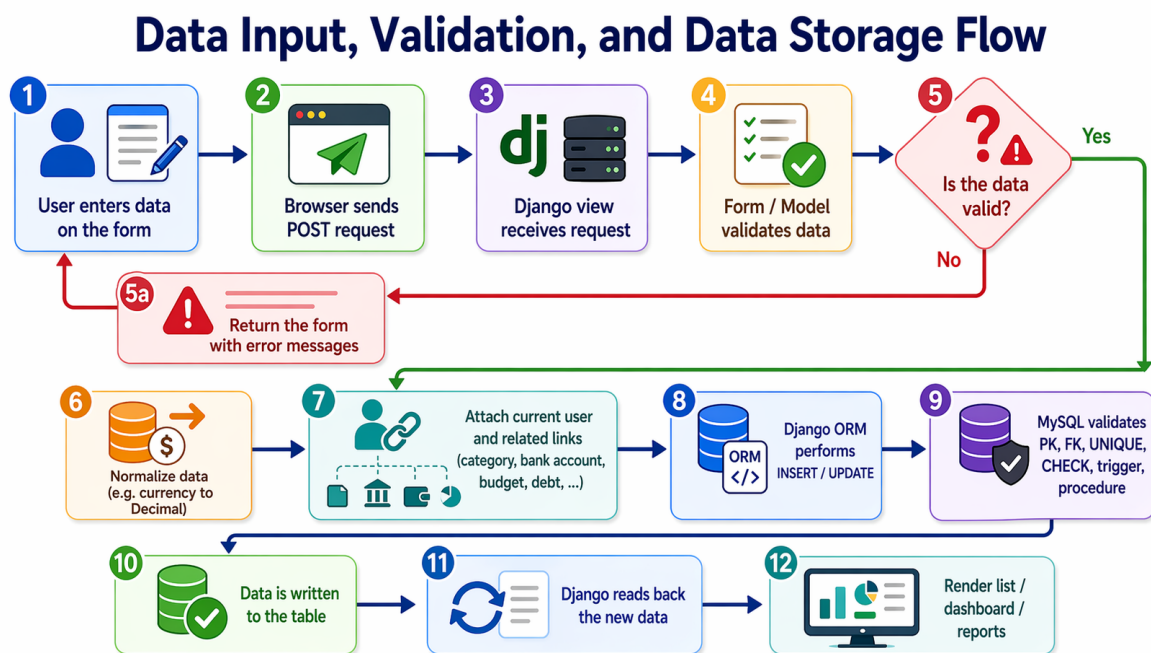


Figure 8: Data flow from input forms to MySQL

One notable point in the integration process is that the system allows users to enter monetary values in a user-friendly format in the interface, for example 1.500.000, while still storing them in the database as standard numeric values. This is handled through the form-processing layer in Django. On the interface side, monetary fields are reformatted to be easy to read and easy to enter. On the backend side, the money string is normalized and converted into a `Decimal` value before being written into numeric columns in MySQL. As a result, the application preserves a friendly input experience without affecting the correctness of financial data in the database.

After the data is saved, the system also reformats monetary values for display in the interface, for example converting them into a format such as 8.500.000 VND. Thus, it can be seen that the interface and the database use two different representations for the same value, but the entire conversion process is controlled at the Django layer to ensure that MySQL always stores standardized and consistent numeric values.

6.3 Flow for updating balances, budgets, and alerts

Not every operation in the application stops at inserting a new record into a table. In financial workflows, each data change usually triggers multiple related effects. For example, when a user creates a new expense, the system not only inserts a record into the **expenses** table, but must also update the account balance, compare the expense against active budgets, and generate alerts if budget usage has exceeded warning thresholds.

In NUFI, this flow is implemented through cooperation between Django and MySQL. Django is responsible for receiving requests, coordinating transactions, and associating data with the current user. MySQL handles part of the validation and derived-data updates through triggers and previously installed constraints. After the data is recorded, dashboards and reports can immediately read the latest state from the database.

Figure 9 describes the flow for updating balances, budgets, and alerts after a transaction occurs.

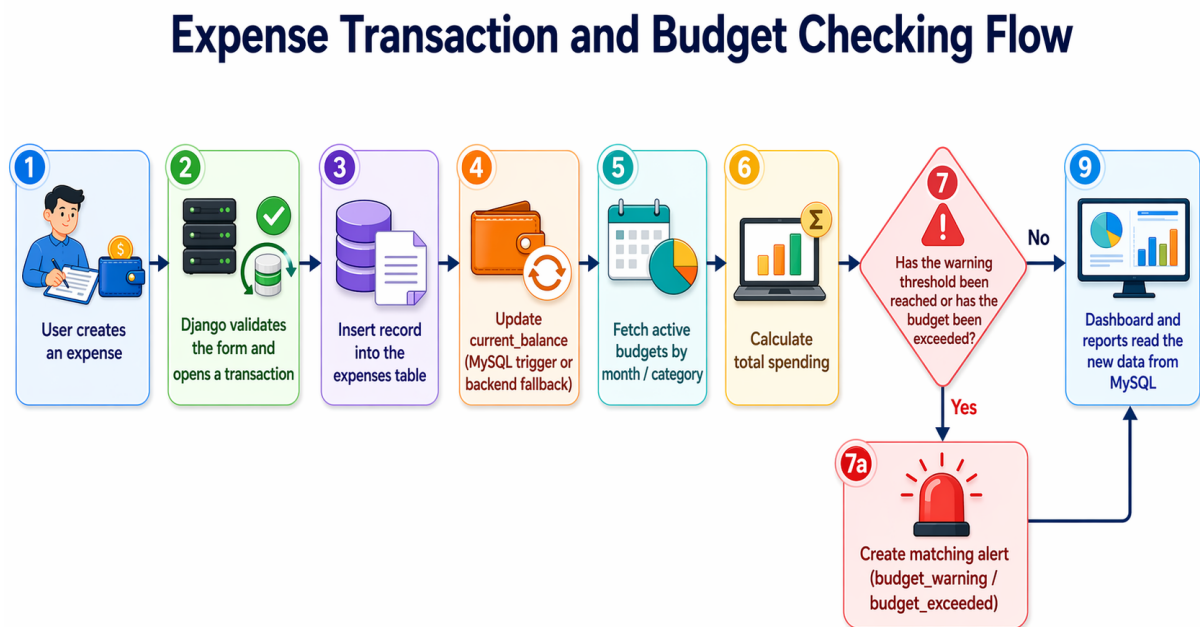


Figure 9: Flow for updating balances, budgets, and alerts

From the diagram above, it can be seen that integrating the database with the application is not simply a matter of reading and writing data, but a coordinated multi-step process to ensure that the data always reflects the user's actual current financial status. This is also evidence of the practical value of the DBMS design presented in previous chapters.

6.4 Illustration of completed features

Based on the database foundation and the Django integration layer, the NUFI system has implemented several main features of the personal finance management problem. First is the group of income and expense management features, which allows users to create, update, soft-delete, and look up financial transactions by time, category, or financial account. Next is the group of financial account management features, which helps users track the current balance, opening balance, and balance history of each account.

In addition, the system supports monthly or category-based budget management, while also generating alerts when a budget is close to or over its limit. The debt management component allows users to track debts, record repayments, and update remaining status. At the extension level, the system also has a sharing module to create groups, add members, and share transactions. Furthermore, the dashboard and reporting components provide an aggregated data-exploitation layer, enabling users to view an overview of their current financial situation and export reports in multiple formats.

Table 6.2 summarizes some of the main system features and their completion status within the scope of the project.

Table 6.2: List of main features and completion status

Feature	Module / Data Used	Status
Personal income management	<code>income;</code> <code>incomes</code> , <code>categories</code> , <code>bank_accounts</code>	Completed within project scope
Personal expense management	<code>expenses;</code> <code>expenses</code> , <code>categories</code> , <code>bank_accounts</code> , <code>alerts</code>	Completed within project scope
Financial account and balance management	<code>bank_accounts;</code> <code>bank_accounts</code> , <code>incomes</code> , <code>expenses</code> , <code>debt_payments</code>	Completed within project scope
Budget and alert management	<code>budgets;</code> <code>budgets</code> , <code>expenses</code> , <code>alerts</code>	Completed within project scope
Debt and debt repayment management	<code>debts;</code> <code>debts</code> , <code>debt_payments</code> , <code>alerts</code> , <code>bank_accounts</code>	Completed within project scope
Sharing groups and shared transactions	<code>sharing;</code> <code>groups</code> , <code>group_members</code> , <code>shared_transactions</code>	Completed within project scope

Feature	Module / Data Used	Status
Overview dashboard	dashboard; incomes, expenses, bank_accounts, budgets, alerts	Completed within project scope
Reports by month, year, category, and trend	reports; incomes, expenses, bank_accounts, budgets	Completed within project scope
Excel / PDF report export	reports; aggregated data from the main financial tables	Completed within project scope

To illustrate the user-interface level, one or two representative screenshots may be inserted in this section. The two most suitable screenshots are the overview dashboard and the income or expense management interface, because they clearly reflect the relationship between database data and the application display.

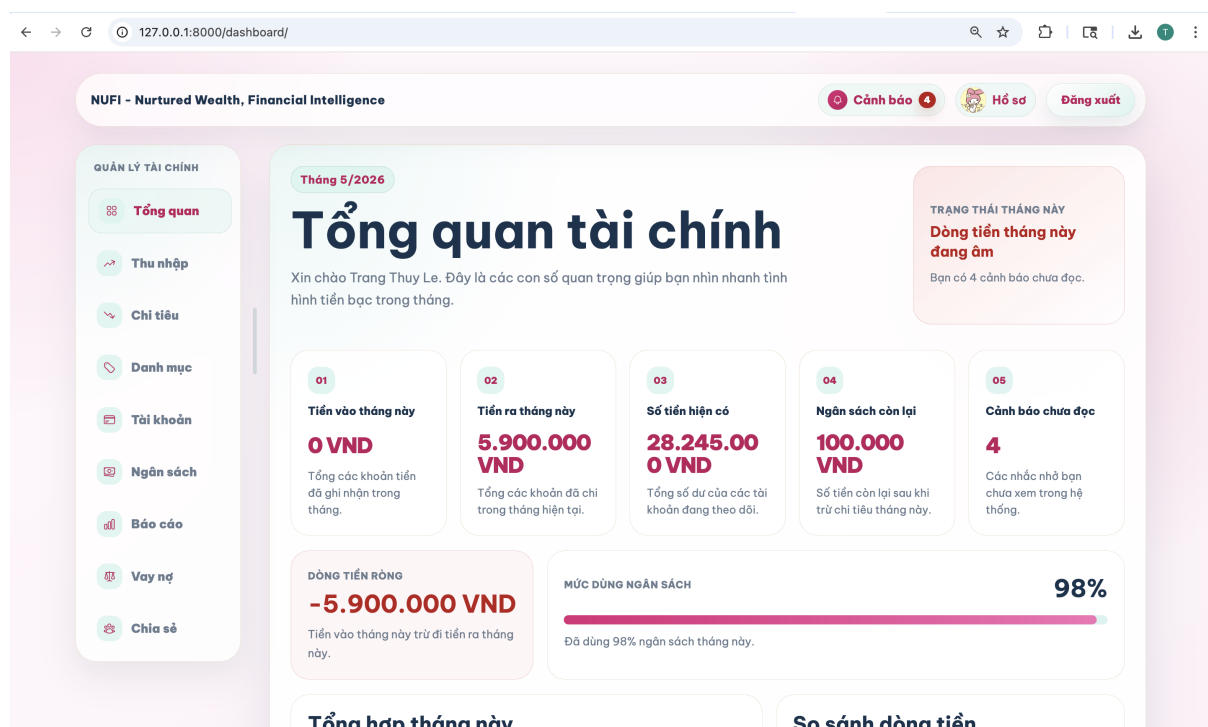


Figure 10: Overview dashboard of the NUFI system

Figure 10 shows that aggregate data such as monthly income, monthly expenses, account balances, remaining budgets, and unread alerts are all retrieved directly from the database and then processed again in the backend before being displayed.

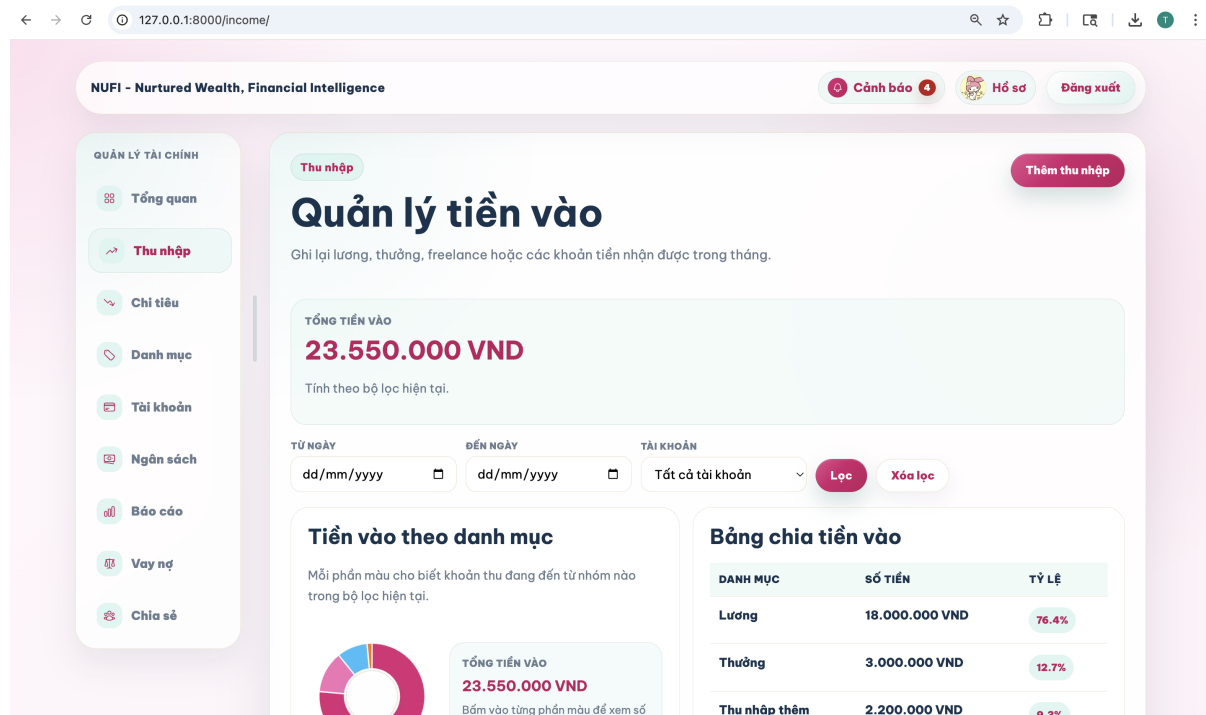


Figure 11: Income and expense management interface of the NUFI system

Figure 11 illustrates the transaction input and lookup functions. This is where users interact directly with financial data, while Django and MySQL cooperate to validate, store, update balances, and support future querying.

6.5 Conclusion

Through the integration of the database with the Django application, it can be seen that the NUFI data model does not stop at the design and SQL-implementation level, but has been used in practice within a complete web application. Tables, relationships, indexes, and advanced SQL objects all directly contribute to building functions such as income/-expense management, financial accounts, budgets, debts, sharing groups, dashboards, and reports.

This integration section also shows the clear role of each layer in the system: the interface is responsible for data input and display, Django coordinates the processing flow and context-based checks, and MySQL is responsible for storage and protection of financial data consistency. This is evidence that a well-designed DBMS can absolutely serve as the foundation for a real application with practical usability and extensibility.

7. Extended Section: System Deployment and Operations

If the previous chapters focused on the core of the project, namely the design, implementation, and exploitation of the database, then this chapter only serves as an extension to demonstrate that the NUFI system does not stop at the level of models or SQL scripts, but has been placed in a relatively complete operational environment. The focus of this chapter is not deep infrastructure analysis, but rather to show that the MySQL database and related components can work together with the Django application to form a system that can run locally, be packaged in containers, be deployed on Kubernetes, support scheduled tasks, back up data, and monitor operational status.

In other words, this is evidence of the system's practical value. For the Database Management Systems course, this content is not the main focus like ERDs, relational schemas, triggers, or procedures, but it shows that the data model that has been built can be integrated into a real running system that has backup, monitoring, scheduling mechanisms, and the capability for public demonstration.

7.1 System runtime environments

Within the project, the NUFI system is organized into multiple deployment levels to serve different purposes. At the simplest level, the system can run locally with Django together with MySQL for development, testing, and quick demos. At a higher level, the application is packaged with Docker and run using Docker Compose so that components such as the web app, database, and related services can start synchronously within the same environment [7]. At a more extended level, the project also supports deployment to Kubernetes with Helm to simulate the operation of a real system that can be configured per environment, manage secrets, and separate infrastructure components [8, 9]. In addition, the project also uses GitHub Actions as part of the CI/CD flow to automate checks, tests, and builds, thereby making the deployment process more consistent and reproducible [16].

The existence of multiple deployment levels is not intended to turn the project into an independent DevOps problem, but rather to demonstrate that the designed database can be placed in different runtime environments while still maintaining consistency. In other words, Docker, Kubernetes, Helm, and CI/CD are only extended components that support operation and presentation of the system in a practical context; the main focus of the project remains the design, implementation, and exploitation of the MySQL database. This is especially important for data-management systems, because a good database must not only be correct in terms of modeling, but also be able to operate stably in practice.

To more fully illustrate the system's deployment extension, Figure 12 describes NUFI's overall deployment architecture, from source code and the GitHub Actions / CI-CD

pipeline to local, Docker, and Kubernetes runtime environments.

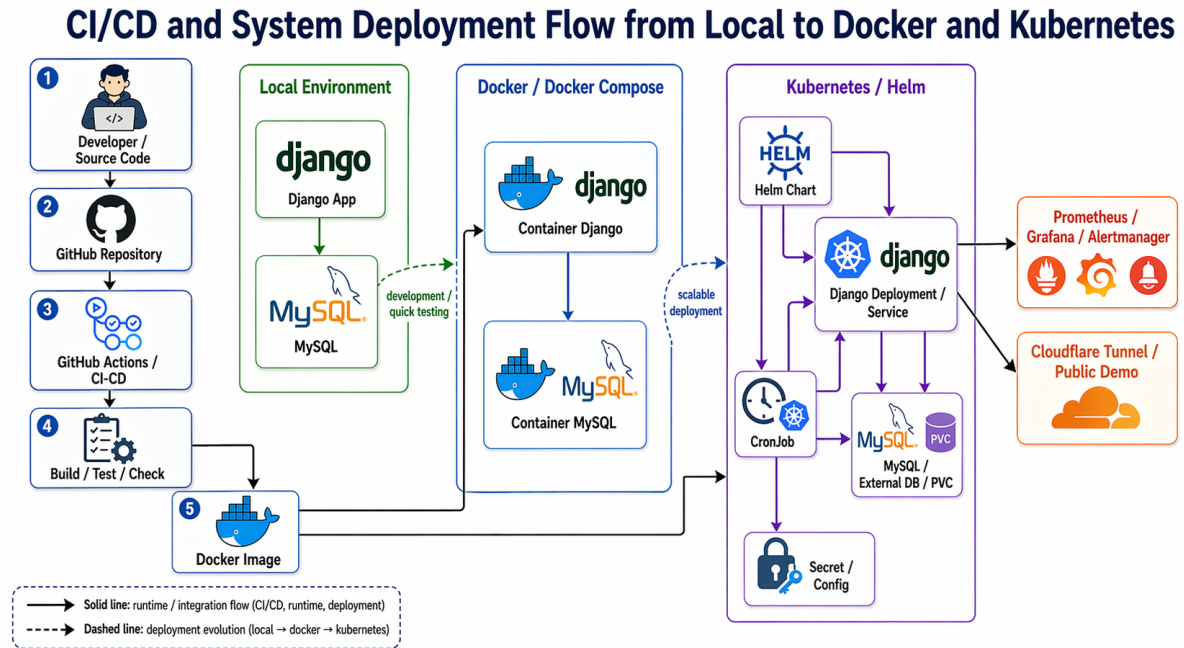


Figure 12: Overall deployment architecture of the NUFi system

Table 7.1 compares the main runtime environments of the system and their respective purposes.

Table 7.1: Comparison of system runtime environments

Environment	Main Components	Purpose
Local	Django development server, local MySQL / Docker DB	Used for development, quick testing, UI checking, and validation of data flows between Django and MySQL.
Docker / Docker Compose	Web container, MySQL container, volumes, and environment variables	Packages the application into a consistent runtime environment, making it easier to reproduce when demonstrating or switching machines.
Kubernetes / Helm	Deployment, Service, Secret, PVC, CronJob, Helm chart	Simulates a more realistic operational environment, supporting environment-based configuration, secret management, and structured deployment.
Public demo	Kubernetes / local service combined with Cloudflare Tunnel	Allows the demo to be published externally without directly exposing internal infrastructure.

7.2 System packaging and deployment

From a deployment perspective, Django is the main entry point of the application, while MySQL is the central data-storage component. When running locally, developers can directly start the application with the development server to check business flows and CRUD operations. However, when a more realistic environment is needed, Docker and Docker Compose are used to package the components, allowing the entire system to be started with a consistent configuration rather than depending on manual setup on each machine.

At a higher level, Kubernetes and Helm are used as an extension to demonstrate that the system can be deployed following modern infrastructure patterns. In this organization, Helm plays the role of grouping Kubernetes resources into a reusable deployment package, making it easier to manage application configuration, the database, secrets, volumes, and cronjobs. Although this part is not the main content of the DBMS course, it shows that the database designed in the project is stable enough to serve as the foundation for a system capable of deployment and expansion.

In addition to manual deployment, the project also uses GitHub Actions as part of the CI/CD flow to automate several checking and deployment steps [16, 17, 18]. Specifically, the CI/CD pipeline is used to run source-code checks, execute tests in appropriate environments, build Docker images, and support the deployment flow with Helm on Kubernetes. Within the scope of this project, this component is not the main focus like database design or SQL implementation, but it shows that the system has been organized in a way that is reproducible, testable, and deployable through a clear process.

In addition, the project also supports Cloudflare Tunnel to publish the demo externally [13]. The role of this component is not in data management, but in creating conditions for the system to be presented, tested, and shared more conveniently without requiring deep intervention in network configuration or public domains.

7.3 CronJob, backup, and monitoring

A real-world data management system not only needs to store and query data, but also needs operational support components such as scheduled tasks, backups, and monitoring. In NUFI, Django management commands are used to perform periodic tasks such as budget alert checks, overdue debt checks, daily report generation, and data backup. When running on Kubernetes, these tasks are attached to CronJobs so that the system can operate automatically according to schedule rather than depending on manual intervention.

In terms of backup, the project supports both report/export backup and MySQL-related data backup in the form of dumps or artifacts generated at runtime. The significance of

this mechanism lies in protecting data and creating conditions for recovery when necessary. Although restore functionality is only basic within the scope of the project, having periodic backups is still an important demonstration of practical database administration thinking.

In addition, the system is monitored by tools such as Prometheus, Grafana, and Alertmanager [10, 11, 12]. These components help observe application status, track health checks, collect operational metrics, and send alerts when abnormal signs appear. In the context of the project, monitoring is not intended to replace business alerts in the database, but rather to complement them at the infrastructure and operations level.

To illustrate the relationship among scheduled tasks, backup, and monitoring, Figure 13 describes the overall operational flow of these extended components.

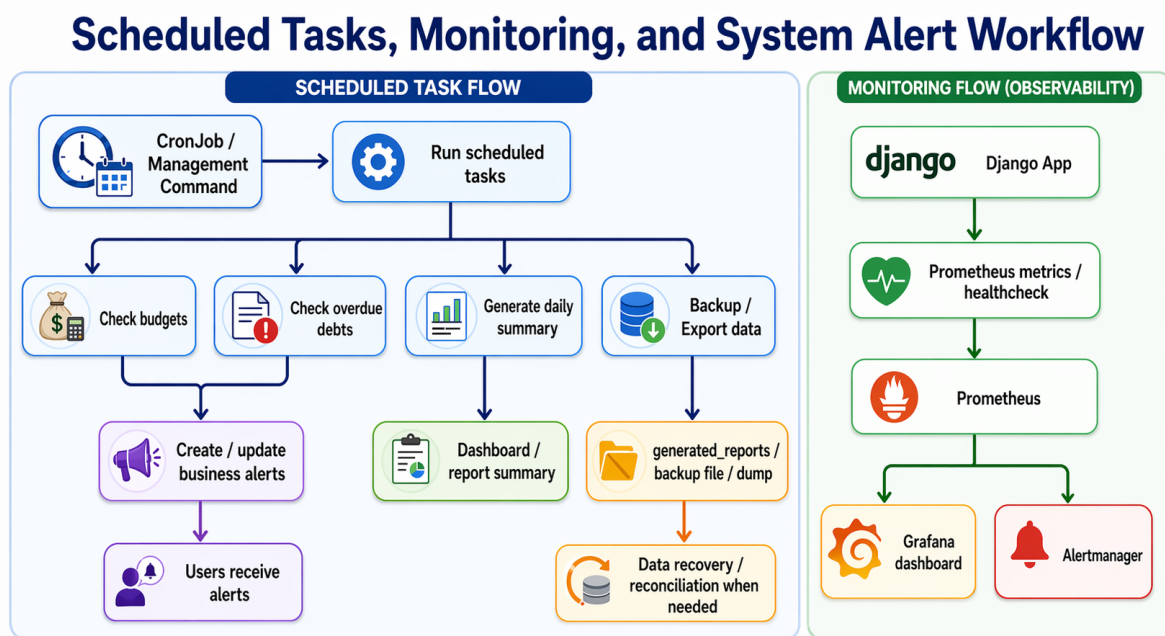


Figure 13: CronJob, backup, and monitoring flow of the NUFI system

7.4 Roles of extended components

Although they are not the main focus of the thesis, the deployment and operational components still have clear demonstrative value for the NUFI system. They show that the designed database is not only useful for theoretical examples, but can also operate within an application system that has configuration, backup, background tasks, and operational status monitoring.

Table 7.2 summarizes the role of several extended components used in the project.

Table 7.2: Roles of extended deployment and operational components

Component	Role in the System
Docker	Packages the application and related components into a consistent runtime environment, making the system easier to reproduce for testing and demos.
Docker Compose	Orchestrates multiple containers in the local/dev environment, especially useful when Django, MySQL, and supporting services need to run together.
Kubernetes	Provides a more structured deployment environment, supporting management of workloads, services, volumes, secrets, and scheduled tasks.
Helm	Helps package Kubernetes deployment configuration into reusable charts, convenient for environment-based configuration.
CronJob	Automates scheduled tasks such as budget checks, overdue debt checks, backup, and summary report generation.
Backup	Protects data and exported files, providing a basis for system recovery when needed.
Prometheus / Grafana / Alertmanager	Monitor the operational state of the application and infrastructure, display technical dashboards, and send alerts when abnormal signs are detected.
Cloudflare Tunnel	Helps publish the demo externally quickly without directly exposing internal infrastructure.
GitHub Actions / CI-CD	Automates checks, tests, image builds, and supports system deployment through a more consistent process.

Overall, the combination of containerization, orchestration, scheduled tasks, monitoring, and CI/CD helps the NUFI system move closer to a realistic operational workflow, even though this is only an extended illustration rather than the main focus of the DBMS thesis.

7.5 Conclusion

In summary, the deployment and operations section of the NUFI project is only an extension intended to demonstrate the practical operability of the system, rather than the central focus of the DBMS thesis. The value of this chapter lies in showing that the MySQL database, SQL scripts, and Django integration layer are complete enough to operate in multiple environments, from local execution to containerization and orchestrated

deployment.

Thanks to components such as Docker, Kubernetes, Helm, CronJob, backup, and monitoring, the NUFI system is not merely a data model described on paper, but has moved closer to an application that can actually run, be tested, and be observed. This helps reinforce the practical value of the entire project, even though its core remains database design and database management.

8. Testing and Evaluation

After completing the design, SQL implementation, and integration with the Django application, the next step is testing to confirm that the system works correctly both at the data level and at the business level. In the NUFI project, testing does not stop at basic CRUD operations, but extends to data aggregation, integrity constraints, advanced SQL objects such as **VIEW**, **FUNCTION**, **PROCEDURE**, and **TRIGGER**, as well as business rules related to budgets, debts, alerts, and group sharing.

An important point is that the system's test suite is executed directly on a real MySQL environment instead of using a simulated SQLite environment. This makes the test results more accurately reflect the real behavior of the triggers, functions, procedures, and views implemented in the database. In the current testing round, the system successfully passed all **22 tests** using the `manage.py test` command on MySQL and returned the result **OK**. Therefore, the evaluation in this chapter can be based on actual test data rather than theoretical description alone.

In addition to the automated test suite, the system was also run in practice to verify the integration between Django, MySQL, and the user interface. After the latest update, the system was rebuilt and verified on both Docker Compose and Kubernetes environments. Smoke tests passed on `http://127.0.0.1:8001` and `http://127.0.0.1:8002`, confirming that the deployed versions were reachable and operational. Figure 14 is a screenshot demonstrating that the system is running after full integration of the main components.

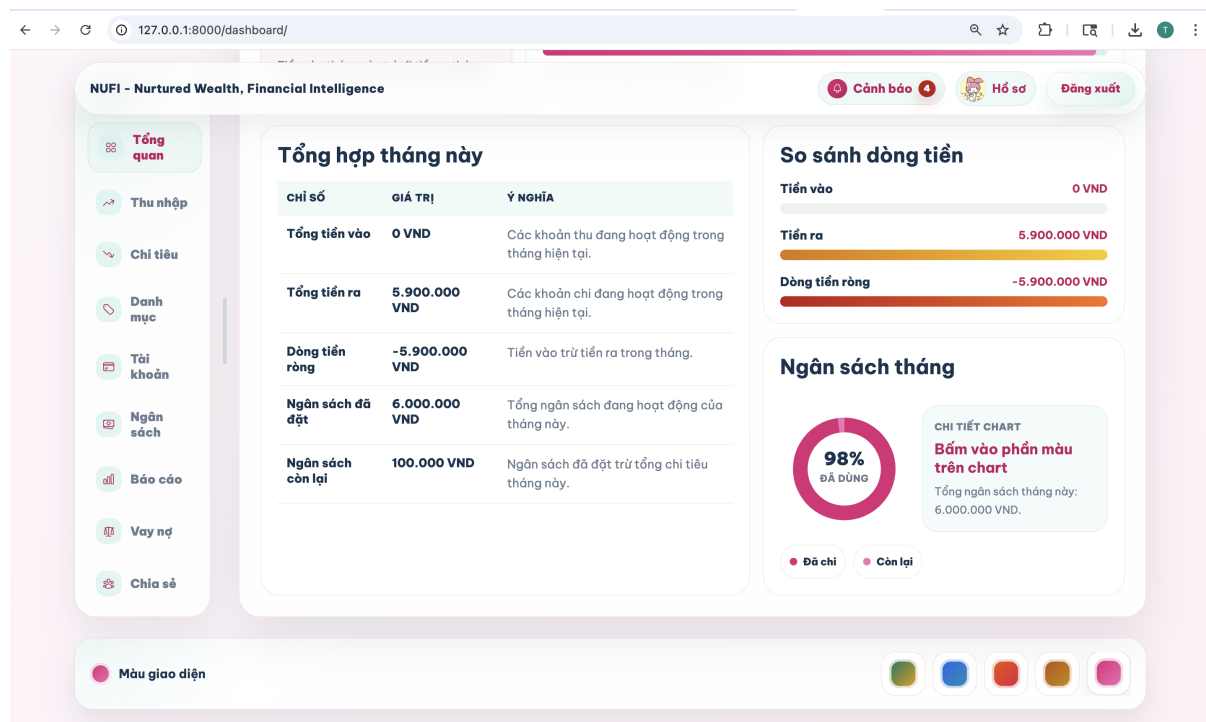


Figure 14: Screenshot showing the NUFI system running after integration

8.1 Functional and business testing

Functional testing focuses on the main business flows of the system, including adding transactions, updating balances, calculating budget usage, debt repayment, alert generation, ownership-based access control, and shared transactions within groups. This is an important testing layer because it shows that the database design truly supports the application's functions.

In the current test suite, *insert* and *update* operations are tested directly through flows such as creating an expense, creating a budget, repaying a debt, and sharing a transaction. Aggregate queries are also tested through the monthly report and overview report screens with selected date ranges. In addition, incorrect ownership-access cases, such as when a user attempts to access another user's data, are also tested to ensure data safety at the application layer.

Table 8.1 summarizes representative functional test cases that were actually executed and their corresponding results.

Table 8.1: **Functional test cases and results**

Test Group	Actual Test Case	Expected Result	Result
Expenses	Create a new expense belonging to the correct user, category, and bank account	The expense record is created, the account balance is reduced accordingly, and a budget alert is created when the threshold is reached	Pass
Expenses	Expense list of one user when the system contains data for multiple users	Only expenses belonging to the currently logged-in user are displayed	Pass
Data ownership	User A opens the edit page of User B's expense	The system denies access and returns 404	Pass
Budgets	Create a budget with overall scope but submit a category as well	The system automatically removes category and stores the overall budget with the correct structure	Pass
Budgets	Display the budget list when multiple expenses already exist in the same period	Correctly compute used_amount , remaining_amount , and usage_percent	Pass
Data ownership	User A opens the edit page of User B's budget	The system denies access and returns 404	Pass
Debt	Create a valid debt payment	Correctly reduce remaining_amount , update status , and adjust account balance at the same time	Pass
Debt and alerts	Run the debt_overdue_check command for an already overdue debt	The debt status changes to overdue and an overdue-debt alert is created	Pass
Data ownership	User A opens the debt_detail page of User B	The system denies access and returns 404	Pass

Test Group	Actual Test Case	Expected Result	Result
Reports	Open the monthly report when income, expenses, and budgets already exist	Correctly compute total inflow, total outflow, difference, budget, and available funds	Pass
Reports	Open the overview report with a limited date range	Only aggregates data that falls within the selected date range	Pass
Group sharing	A non-member accesses group details	The system denies access and returns 404	Pass
Group sharing	An active member shares one of their own expenses into the group	A <code>shared_transactions</code> record is created correctly with a valid <code>expense_id</code>	Pass

From the table above, it can be seen that the main business operations of the system have been tested at a relatively comprehensive level. In particular, many tests do not merely check whether data is saved, but also verify follow-up effects such as balance updates, alert generation, budget usage calculations, and blocking unauthorized access. This matches the characteristics of a financial management system, where the value of data lies not in individual records but in the consistency of the entire business flow.

8.2 Testing SQL objects and integrity constraints

In addition to tests that go through the Django layer, the system also has a separate group of tests to verify SQL objects implemented directly in MySQL. This part is especially important for the Database Management Systems course because it proves that `VIEW`, `FUNCTION`, `PROCEDURE`, and `TRIGGER` are not just defined on paper, but actually run in a real database environment.

In the current test suite, the files `functions.sql`, `views.sql`, `procedures.sql`, and `triggers.sql` are loaded into the test database before testing begins. The system then calls these SQL objects directly through cursors to verify returned values and their reactions to valid or invalid data cases. This approach makes it possible to evaluate real MySQL behavior rather than inferring it only through the ORM layer.

Table 8.2 presents representative test cases for the SQL-object group and their results.

Table 8.2: **SQL object test cases**

SQL Object Type	Object Under Test	Test Content	Result
FUNCTION	<code>fn_monthly_net_amount</code>	Given previously created income and expense data, the function must return the correct monthly income-expense difference	Pass
VIEW	<code>vw_budget_usage</code>	Must return correct <code>amount_used</code> , <code>remaining_amount</code> , <code>usage_percent</code> , and <code>usage_status</code> of budgets	Pass
PROCEDURE	<code>sp_monthly_summary</code>	Must return correct total income, total expenses, net value, unread alert count, and total balance of active accounts	Pass
TRIGGER	<code>trg_alerts_bi_validate</code>	When creating an alert with multiple related objects at the same time, the trigger must reject it and return an error	Pass

The test results show that the system’s SQL objects work as expected in the tested cases. In particular, the trigger’s rejection of invalid data demonstrates that the database-level data-protection layer is functioning effectively. Likewise, the correct results from procedures and functions reinforce that MySQL-based aggregation logic can be used directly for reports and dashboards.

8.3 Evaluation of achieved results

From the results of functional testing and SQL-object testing, it can be seen that the NUFIS system has reached a relatively good level of completeness both at the application layer and at the database layer. The notable point is not only that the main functions work, but also that many important business rules have been verified in a real MySQL environment. This gives the evaluation results more value than in cases where only Django logic or a simplified SQLite environment is tested.

At the business level, important flows such as creating expenses, updating account balances, tracking budgets, generating alerts, repaying debts, updating debt status, and sharing transactions within groups all produced results matching expectations. The test suite not only confirms that data is saved successfully, but also confirms the side effects across multiple related tables. This is a strength of the system, because in a personal

finance management problem, the value of the database does not lie in each isolated table but in the consistency of the entire processing chain.

At the DBMS level, the system has demonstrated that advanced SQL objects do not only exist at the design level, but actually operate in a real runtime environment. Direct tests with `fn_monthly_net_amount`, `vw_budget_usage`, `sp_monthly_summary`, and the alert-validation trigger show that the database is capable of storing, calculating, and protecting business constraints at the same time. A clear example is that the result of `sp_monthly_summary` correctly reflects account balances after the income and expense triggers have already taken effect, rather than returning only the initial balance. This proves that the data logic inside MySQL has truly been activated and verified.

At the system-integration level, the fact that the test suite runs successfully directly on MySQL with the result `22 tests OK` shows that the data model, Django backend, and SQL objects can work together stably in a real environment. This is academically valuable for a DBMS project because it proves that the entire process from ERD design, relational-schema transformation, SQL implementation, application integration, and testing has been connected into one complete chain.

Compared with manual financial management, the NUFi system shows clear advantages in three aspects. First is the accuracy and consistency of data, thanks to constraints, triggers, and ownership checks. Second is the ability to aggregate and exploit data, expressed through dashboards, monthly reports, date-range reports, and financial calculations. Third is its extensibility and operational capability, because the system does not stop at basic CRUD but already includes backup, cronjobs, and practical deployment environments.

In addition, the Excel export feature was improved using `openpyxl`. Exported reports now include a report title, KPI summary cards, formatted currency values, styled table headers, frozen panes, and autofilter support. This makes exported files more readable and closer to practical financial reports instead of raw tabular dumps.

Table 8.3 briefly compares manual financial management with using the NUFi system.

Table 8.3: Comparison between manual management and using the NUFi system

Criterion	Manual Management	Using NUFi
Recording income and expenses	Easy to forget, easy to miss entries, hard to synchronize	Input forms are available, data is stored centrally in the database, and historical lookup is easier

Criterion	Manual Management	Using NUFI
Tracking account balances	Usually requires manual calculation	The system automatically updates balances based on transactions and debt payments
Budget control	Hard to know how much has been spent, easy to exceed limits	Supports monthly/category budgets and automatic alerts
Debt tracking	Easy to forget due dates or remaining amounts	Provides debt status, repayment history, and overdue alerts
Summary reporting	Time-consuming to aggregate manually	Includes dashboard, monthly reports, date-range reports, and formatted Excel/PDF exports
Data safety	Personal data is easy to mix up and hard to control access to	Data is tied to each user, with ownership control in both backend and database

From the overall evaluation perspective, the system has achieved the objective of building a personal finance management platform with tightly structured relational data, SQL exploitability, application integration, and validation through actual testing. This is a positive result both academically and practically.

8.4 Current limitations and improvement directions

Although the system has achieved relatively good results, the project still has several limitations at both the testing level and the deployment level. Pointing out these limitations is necessary because it helps evaluate the system more objectively and clarifies future improvement directions.

In general, the current limitations mainly stem from three causes. First, the scope of the DBMS project prioritizes the data model, SQL objects, and core business operations, so some extended components have not been developed as deeply as the database part. Second, implementation time was concentrated on the most frequently used flows such as income, expenses, budgets, debts, and reports, so test coverage for some secondary flows is not yet fully comprehensive. Third, some limitations come from the execution environment itself, such as differences between MySQL capabilities and certain framework-level constraints.

Table 8.4 presents the main limitations of the system in terms of limitation, cause, miti-

gation direction, and level of impact.

Table 8.4: **Current limitations, causes, and improvement directions**

Limitation	Cause	Improvement Direction	Impact Level
Incomplete coverage of soft-delete flows and some edge cases	The current test suite prioritizes <i>create</i> , <i>update</i> , <i>reporting</i> , and <i>ownership</i> flows because these are the system's core business operations	Add dedicated tests for soft delete, restore, boundary values, and invalid-data scenarios	Medium
Not all SQL objects are directly tested	Testing has so far prioritized representative objects with strong business impact such as the main function, view, procedure, and trigger	Extend direct testing to remaining aggregate views, transaction-creation procedures, and other validation triggers	Medium
Post-deployment testing has not been unified into one complete suite	Kubernetes, back-up/restore, and monitoring are considered extensions rather than the DBMS core of the project	Add smoke tests and integration tests for deployment environments, cron-jobs, backups, and monitoring	Low
Running tests on MySQL depends on test-user permissions	Django needs to create a separate test database, while some normal MySQL users only have permissions on the main database	Create a dedicated test user for CI or local environments with limited but sufficient permissions to create and drop test databases	Low
Some framework compatibility warnings still appear on MySQL	Some third-party libraries use constraint mechanisms that MySQL does not fully support, such as conditional unique constraints	Move part of the validation to the application layer or replace it with equivalent constraints better suited to MySQL	Low

Limitation	Cause	Improvement Direction	Impact Level
No mobile version for faster real-world usage	The project currently focuses on the web app in order to highlight the database and Django integration	Develop a mobile app using the same API and data layer so the system better matches everyday usage habits	Medium

From the limitations above, it can be seen that most of the current issues do not lie in an incorrect data model or an inability of the system to run, but mainly in test coverage, completeness of extended components, and the deployment scope within the project boundary. This also shows that the current foundation of the system is still good enough for further development in the future.

8.5 Conclusion

From the testing results presented above, it can be affirmed that the NUFI system has reached a relatively good level of completeness within the project scope. The test suite not only verifies the main functions at the application layer, but also directly tests several important SQL objects in MySQL, thereby proving that the system's database design is feasible and works correctly in a real environment.

Although some limitations remain, such as incomplete coverage of delete flows or not yet testing every SQL object separately, the current results are still sufficient to conclude that the system has achieved its objectives well. This also serves as a basis for affirming that the value of the project lies not only in data modeling, but also in its ability to be deployed, integrated, and validated through actual experimentation.

9. Conclusion and Future Development

Through the processes of analysis, design, implementation, and testing, the project has built a personal finance management system with a relatively complete relational database and practical operational capability. The NUFI system has modeled the important entities of the problem such as users, financial accounts, income, expenses, budgets, alerts, debts, debt repayments, and sharing groups; it has also transformed them into a clear relational schema, normalized the data, implemented the database with MySQL, and integrated it with a Django application.

From the DBMS perspective, the main value of the project lies in going beyond simple table design to exploit advanced DBMS capabilities such as primary keys, foreign keys, integrity constraints, indexes, views, functions, procedures, and triggers. Thanks to this,

the database not only plays the role of data storage, but also directly participates in ensuring data correctness and supporting important financial business operations such as balance updates, budget tracking, debt management, and alert generation.

In addition to the academic meaning within the Database Management Systems course, the project also demonstrates the practical value of combining SQL with a complete web application. When integrated with Django, the designed data model can directly support features such as data entry, lookup, dashboard aggregation, reporting, data sharing, and business testing. This proves that a good database design can fully become the foundation of a software system that is usable in practice.

However, the project still has several limitations. First, the current feature scope focuses mainly on core personal finance operations and has not yet been extended deeply into areas such as recurring transactions, intelligent financial analysis, or synchronization with external data sources. Second, although the test suite already covers many key business flows and several important SQL objects, it still does not cover all edge cases or the entire list of SQL objects in the system. In addition, some extended parts such as backup/restore, monitoring, and Kubernetes operations currently remain at the demonstration level rather than being the core objective of the DBMS project.

In the future, the system can be developed further in several directions. First, support for *recurring income* and *recurring expense* can be added to better manage monthly repeated transactions. Second, the reporting and financial analysis part can be extended with features such as cash-flow forecasting, spending-trend analysis, or budget optimization suggestions. Third, the alert system can be improved and integrated with additional channels such as email or real-time notifications. Fourth, public deployment capability can be improved further, with better scaling, stronger monitoring, backup, and recovery mechanisms. Fifth, collaborative and sharing modules can be expanded so that the system supports groups of users who manage shared budgets or collective expenses more effectively.

In addition, another important direction is to expand the system toward mobile applications. In the current context, users tend to prefer *mobile apps* over web apps for everyday personal-management problems, because mobile devices are more convenient for quick data entry, instant tracking, and real-time notifications. For personal finance management in particular, the ability to record income or expenses right at the moment they occur is very important, so developing a mobile version in the future would make the system better aligned with real usage habits. At that point, the existing database, API, and business rules of NUFI could still be reused as the shared foundation, while the mobile application would serve as an extended interaction channel for users.

In summary, the project has achieved its goal of building a personal finance management system with a solid data foundation, aligned with the orientation of the Database Man-

agement Systems course, and capable of being integrated into a real application. This result not only confirms the central role of databases in information systems, but also shows that combining DBMS design with application deployment is an approach with value both academically and practically.

References

- [1] Ramez Elmasri and Shamkant B. Navathe, *Fundamentals of Database Systems*, 7th Edition, Pearson, 2021.
- [2] Abraham Silberschatz, Henry F. Korth, and S. Sudarshan, *Database System Concepts*, 7th Edition, McGraw-Hill, 2019.
- [3] Oracle, *MySQL 8.0 Reference Manual*, <https://dev.mysql.com/doc/refman/8.0/en/>,
- [4] Django Software Foundation, *Django Documentation*, <https://docs.djangoproject.com/en/5.2/>,
- [5] Django Software Foundation, *Django Documentation – Models and Databases*, <https://docs.djangoproject.com/en/5.2/topics/db/>,
- [6] Django Software Foundation, *Django Documentation – Forms*, <https://docs.djangoproject.com/en/5.2/ref/forms/>,
- [7] Docker Inc., *Docker Documentation – What is Docker?*, <https://docs.docker.com/engine/docker-overview/>,
- [8] The Kubernetes Authors, *Kubernetes Documentation*, <https://kubernetes.io/docs/>,
- [9] The Helm Authors, *Helm Documentation – Introduction*, <https://helm.sh/docs/intro/>,
- [10] The Prometheus Authors, *Prometheus – Monitoring System and Time Series Database*, <https://prometheus.io/>,
- [11] The Prometheus Authors, *Alertmanager Documentation*, <https://prometheus.io/docs/alerting/latest/alertmanager/>,
- [12] Grafana Labs, *Grafana Documentation – About Grafana*, <https://grafana.com/docs/grafana/latest/introduction/>,
- [13] Cloudflare, *Cloudflare Tunnel Documentation*, <https://developers.cloudflare.com/tunnel/>,
- [14] Lê Thị Thùy Trang, *PERSONAL-FINANCE-MANAGEMENT-SYSTEM (NUFI) – Source Code Repository*, GitHub repository, <https://github.com/thtrangnu/PERSONAL-FINANCE-MANAGEMENT-SYSTEM>,

- [15] Lê Thị Thuỳ Trang, *Internal project documents of NUFI: project overview, system requirements, business rules, relational schema, modules, roles & permissions*, the docs/ directory in the project repository, <https://github.com/thtrangnu/PERSONAL-FINANCE-MANAGEMENT-SYSTEM>,
- [16] GitHub, *GitHub Actions Documentation*, <https://docs.github.com/en/actions>,
- [17] GitHub, *Understanding GitHub Actions*, <https://docs.github.com/en/actions/learn-github-actions/understanding-github-actions>,
- [18] GitHub, *Workflow syntax for GitHub Actions*, <https://docs.github.com/actions/using-workflows/workflow-syntax-for-github-actions>,